B.M.S. COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU

Lab Record

Object-Oriented Modeling – 23CS5PCOOM

Submitted in partial fulfillment for the 5th Semester Laboratory

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Prajwal K S

1BM24CS416

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
August 2025-December 2025

B.M.S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Object-Oriented Modeling(23CS5PCOOM) laboratory has been carried out by Prajwal K S(1BM24CS416) during the 5th Semester August 2025-December 2025

.

Signature of the Faculty Incharge:

Adarsha Sagar

Department of Computer Science and Engineering
B.M.S. College of Engineering, Bangalore

Table of Contents

1. Hotel Management System

Problem Statement
SRS-Software Requirements Specification
Class Diagram Requirements: Minimum 7 classes, 4 attributes and 4 operations in each class, associations, association name, association end names, multiplicity, association class, enumeration, qualified association, aggregation, composition, generalization, ordered, sequencing, multiplicity of attribute, brief description of the Class diagram
State Diagram: one simple state diagram, one advance state diagram (either Sub Machine or Nested State, include any one of the concurrency in your design) Requirements: minimum of 6 states for both the state diagram with state name, do activities, events, guard condition, brief description of State diagram
Use-Case Diagram: Minimum 5 use cases to be identified, and design one simple use case diagram and one advanced use case diagram(ie using include, extend and generalization relationship) and explanation as there in the solution manual.
Sequence Diagram: Write the scenario for any two use case transaction completely. and design the simple sequence diagram with minimum of 5 objects and communication between them. Design advanced sequence diagram using passive and transient objects. Brief explanation for each
Activity Diagram: Design the simple activity diagram showing the algorithm or workflow. Design the advanced activity diagram with swimlanes showing the responsible person for swimlane and also write the brief explanation for both.

1.Hotel Management system

Problem Statement

Running a hotel involves a lot of work managing bookings, assigning rooms, checking guests in and out, handling bills, and keeping track of availability. Doing all this manually often leads to mistakes, double bookings, or delays. Guests get frustrated when their stay is not smooth. A system is needed to make hotel operations faster, easier, and more organized.

SRS

Software Requirements Specification (SRS):

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to outline the requirements and specifications for the development of a Hotel Management System (HMS). It provides a clear understanding of the project objectives, scope, features, and deliverables to ensure alignment between stakeholders, developers, and end-users.

1.2 Scope of this Document

The Hotel Management System will automate and streamline hotel operations, covering reservation management, guest check-in/check-out, room assignment, billing, reporting, and inventory management. This document also defines the estimated development cost, time frame, and technical considerations required for successful implementation.

1.3 Overview

The HMS is a software solution designed to improve hotel efficiency, enhance customer experience, and optimize resource utilization. The system will provide functionalities such as room booking, guest profile management, billing, financial reporting, and integration with external platforms (e.g., third-party booking services).

2. General Description

The Hotel Management System will serve hotel staff, managers, and guests. Staff members can handle reservations, room assignments, billing, and reporting, while guests can manage bookings and payments through an intuitive interface. The system will support different levels of technical expertise and ensure ease of use.

3. Functional Requirements

3.1 Reservation Management

- Allow users to make room reservations online or via the front desk.
- Generate reservation confirmations and send notifications to guests.

3.2 Room Management

- Assign rooms based on availability and guest preferences.
- Track room status (clean, occupied, vacant) in real time.

3.3 Guest Management

- Maintain guest profiles with personal information, preferences, and booking history.
- Facilitate seamless guest check-in and check-out processes.

3.4 Billing and Invoicing

- Generate accurate bills for room charges, services, and taxes.
- Support multiple payment methods (cash, card, digital wallets).
- Generate invoices for corporate clients.

4. Interface Requirements

4.1 User Interface

- Intuitive, user-friendly interface for staff and guests.
- Accessible via web browsers, mobile devices, and desktop applications.

4.2 Integration Interfaces

- Integration with secure payment gateways.
- Integration with third-party booking platforms (e.g., Booking.com, Expedia).

5. Performance Requirements

5.1 Response Time

- The system should respond to user actions within 2 seconds.

5.2 Scalability

- Support at least 1000 concurrent users during peak load.

5.3 Data Integrity

- Ensure accurate and consistent data across all modules.

6. Design Constraints

6.1 Hardware Limitations

- Compatible with standard hotel hardware: computers, printers, POS terminals.

6.2 Software Dependencies

- Relational Database Management System (e.g., MySQL).
- Backend development with Java + Spring Boot.

7. Non-Functional Attributes

7.1 Security

- Implement strong authentication and authorization mechanisms.
- Encrypt sensitive guest and financial data.

7.2 Reliability

- High availability with minimal downtime.
- Fault tolerance in case of failures.

7.3 Scalability

- Support easy expansion to handle more users and hotels.

7.4 Portability

- Cross-platform support: web, Android, iOS.

7.5 Usability

- Clean navigation and easy-to-use interface.

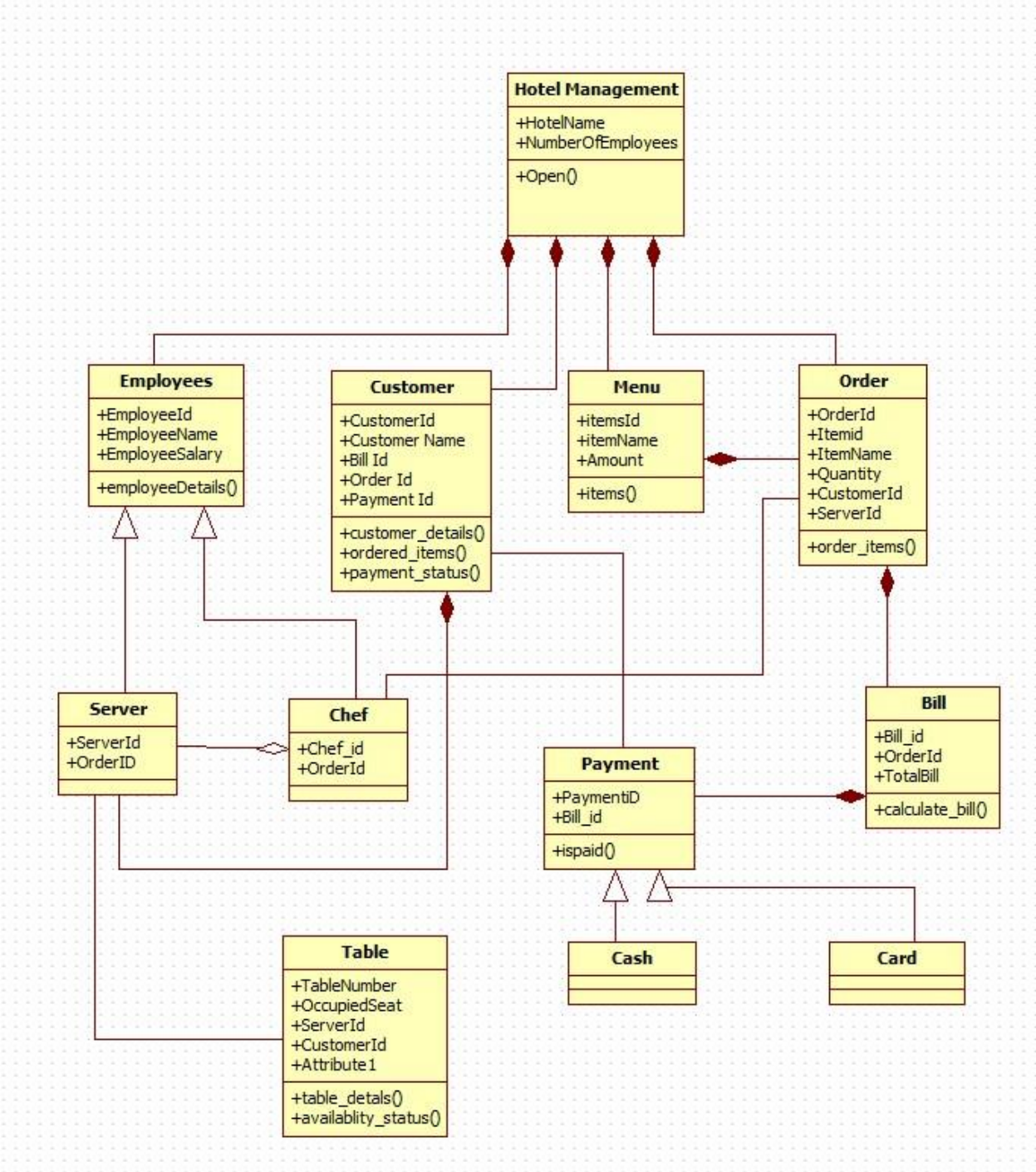
8. Preliminary Schedule and Budget

Estimated Duration: 6 months

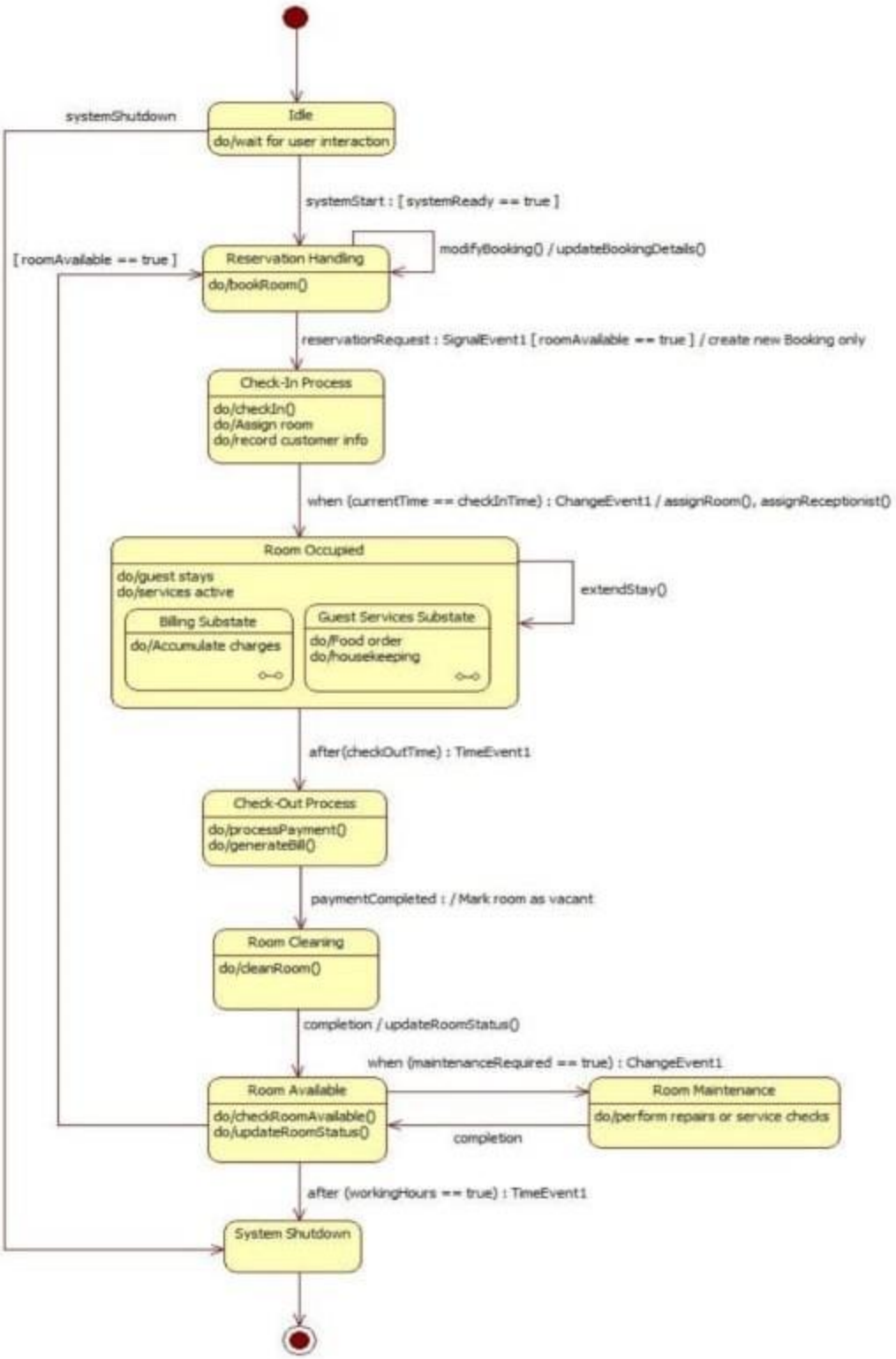
Estimated Budget: $100,000

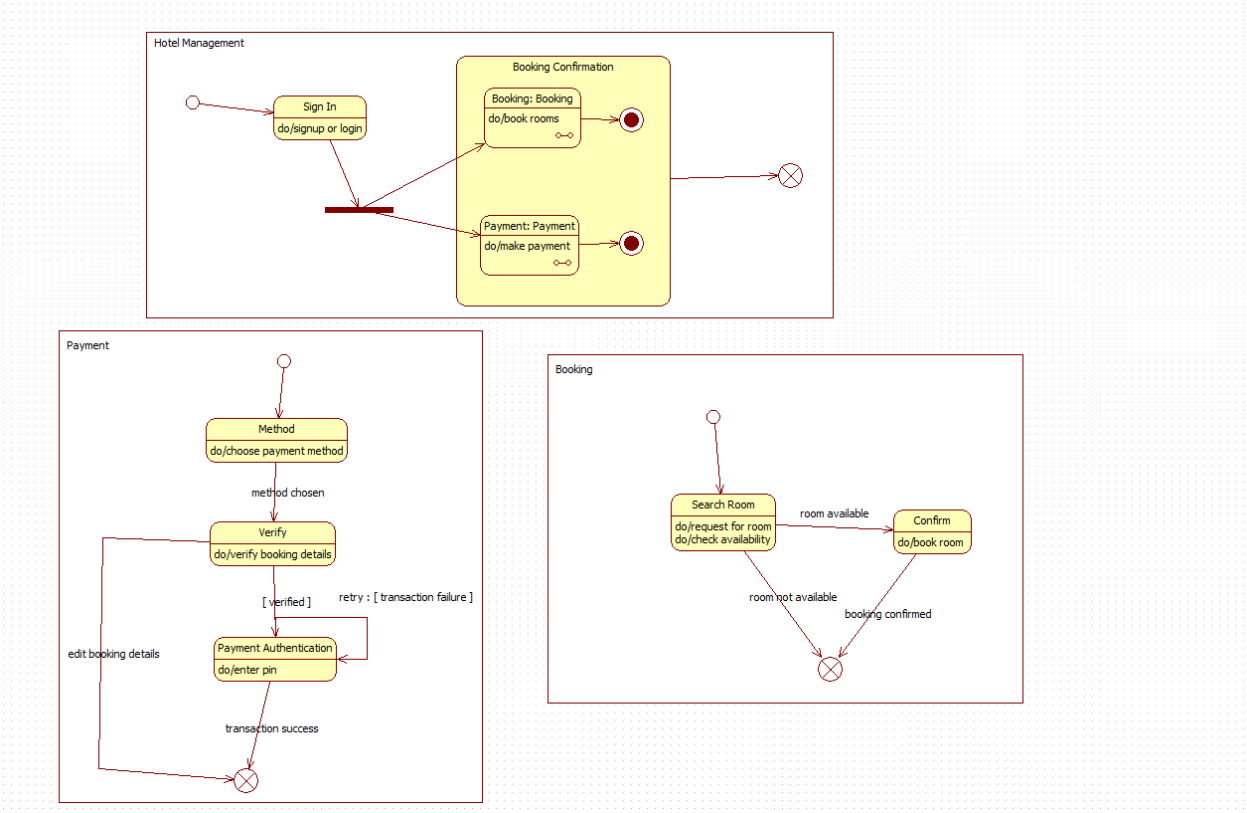
Includes planning, development, testing, deployment, and maintenance.

Class Diagram

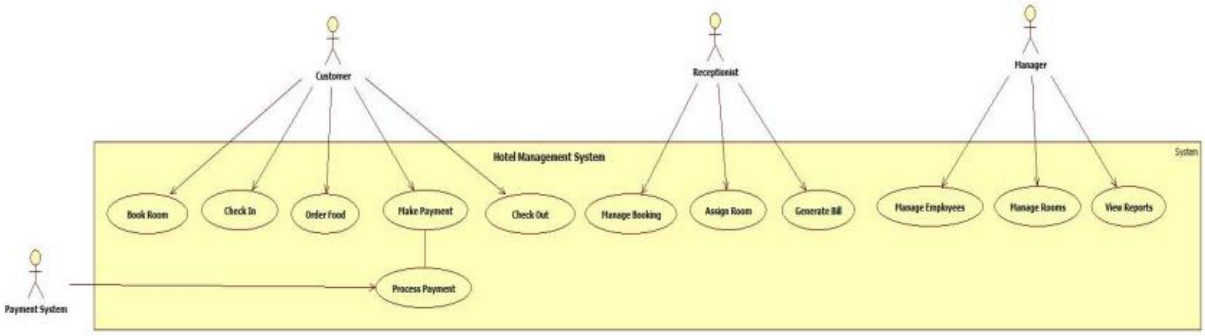


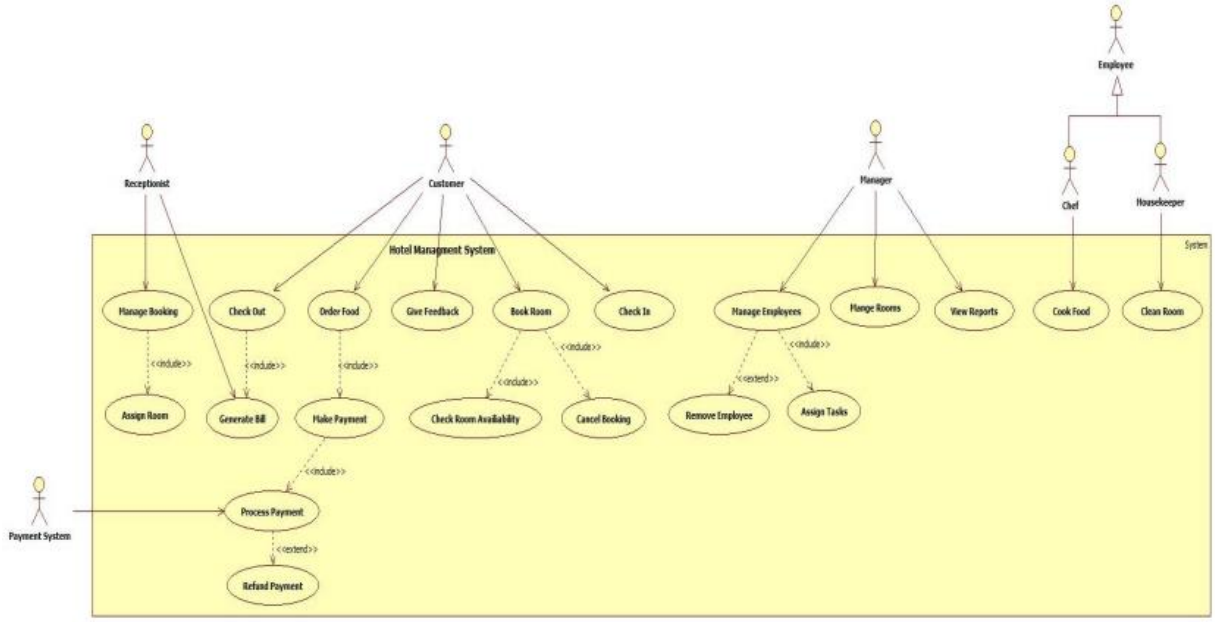
State Diagram



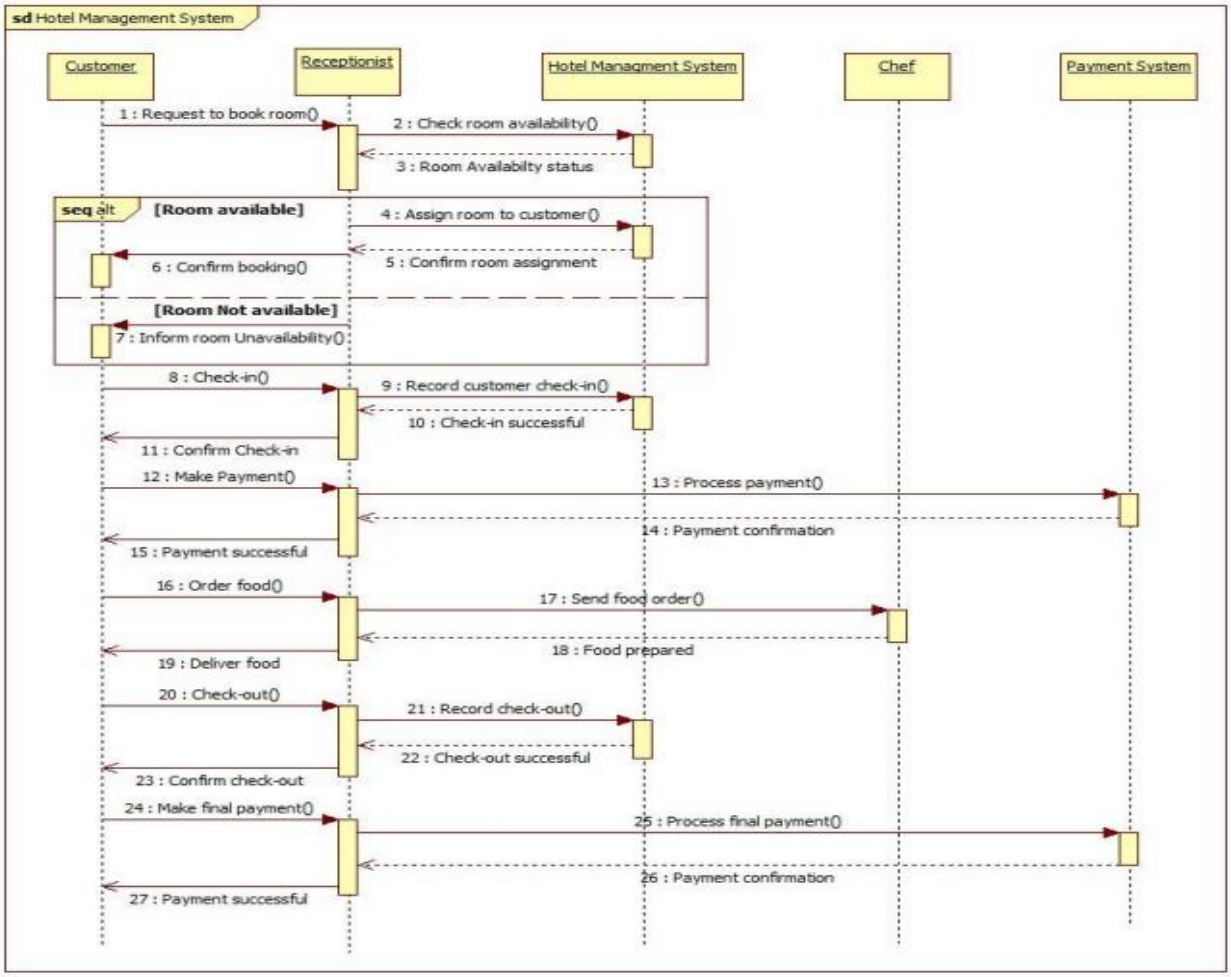


Use Case Diagram

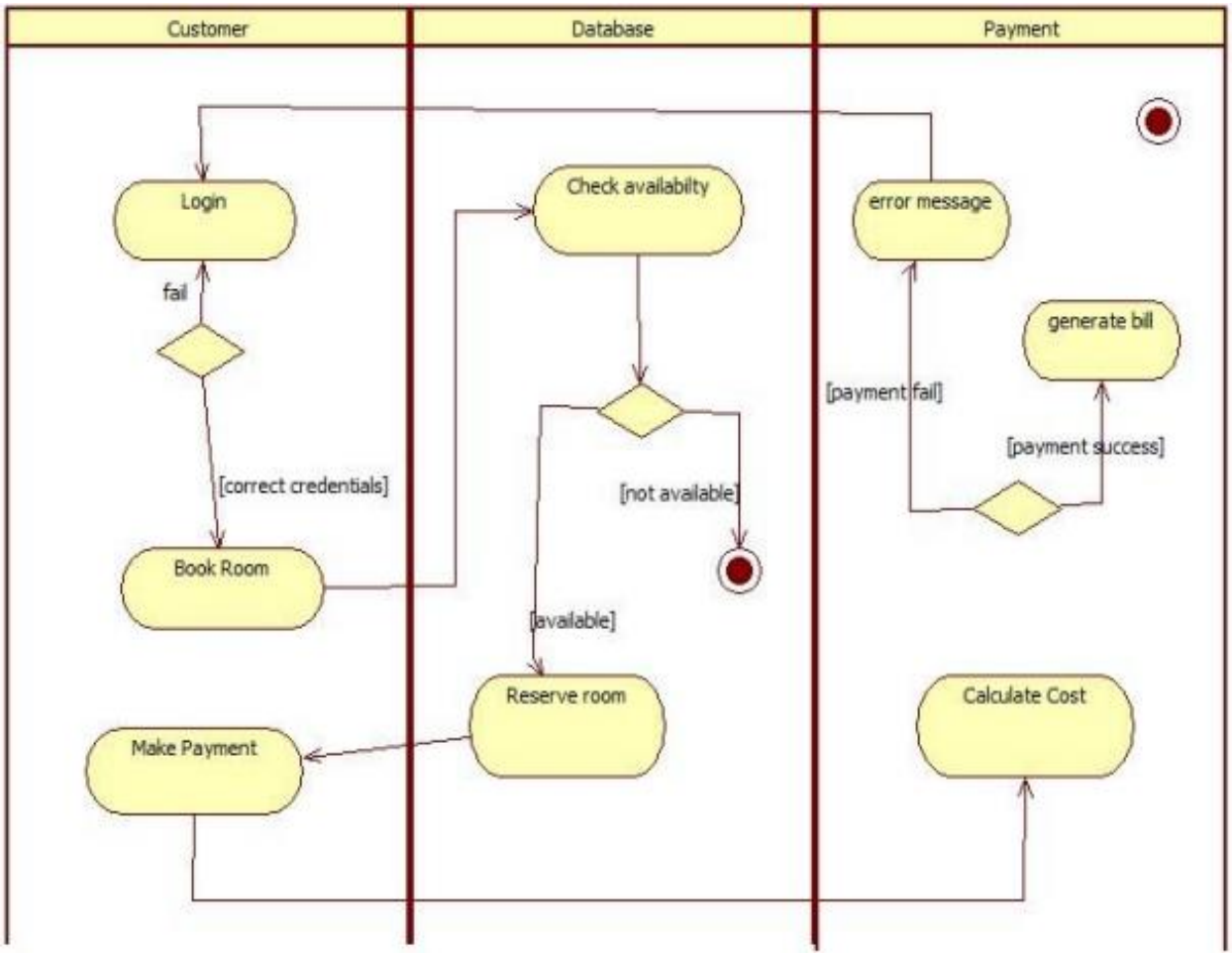




Sequence Diagram



Activity Diagram



2.Credit Card Processing

Problem Statement

Today, people use credit cards everywhere hopping online, paying bills, or swiping at a store. But many businesses face problems with slow payments, errors, and security risks. Customers also get frustrated when transactions fail or take too long. We need a system that can process credit card payments quickly, safely, and without errors.

SRS

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a clear, concise, and detailed Software Requirements Specification (SRS) for the Credit Card Processing System (CCPS). This document defines the functionality, features, constraints, and general system requirements to ensure successful development and implementation of the system.

1.2 Scope of this Document

This document covers all the essential information related to the development of the Credit Card Processing System, including functional and non-functional requirements, user interfaces, design constraints, and other system attributes. The system will be designed to handle credit card transactions securely and efficiently, including authorization, authentication, settlement, fraud detection, and reporting.

1.3 Overview

The Credit Card Processing System is designed to provide a secure and reliable solution for handling financial transactions made via credit cards. It ensures proper communication between merchants, banks, and customers while maintaining compliance with financial regulations and security standards (e.g., PCI-DSS).

2. General Description

2.1 General Functions

The main function of the Credit Card Processing System is to process and secure credit card transactions. The system will:

- Allow merchants to submit card payment requests.
- Authenticate and authorize transactions with issuing banks.
- Handle transaction settlement between customer and merchant accounts.
- Provide fraud detection and risk management.
- Generate transaction reports and financial statements.

2.2 User Characteristics

The Credit Card Processing System will have three distinct types of users:

- Admin: Responsible for system configuration, fraud monitoring, and generating compliance reports.
- Merchant: Submits transactions for authorization, monitors payments, and manages settlements.
- Customer (Cardholder): Uses their credit card for online or POS-based payments.

3. Functional Requirements

3.1 User Registration and Authentication

- Requirement: Merchants and admins must be able to register and log in securely.
- Outcome: Only authorized users can access the system.
- Details: Multi-factor authentication and role-based access must be implemented.

3.2 Transaction Authorization

- Requirement: Validate credit card details and check available balance with the issuing bank.
- Outcome: Transactions will only be approved if valid.

- Details: Use secure protocols to communicate with the bank's system.

3.3 Transaction Settlement

- Requirement: Transfer funds from the customer's bank to the merchant's bank.
- Outcome: Completed transactions are settled accurately.
- Details: Settlement should follow banking regulations and batch processes.

3.4 Fraud Detection and Risk Management

- Requirement: Monitor transactions for suspicious activities.
- Outcome: Fraudulent transactions are flagged or blocked.
- Details: AI-based anomaly detection and blacklisted card verification.

3.5 Reporting and Auditing

- Requirement: Generate transaction history and compliance reports.
- Outcome: Merchants and admins can review past transactions and monitor system usage.
- Details: Reports should be exportable in formats like PDF, Excel, and CSV.

4. Interface Requirements

4.1 Software Interfaces

- Database: Relational database for storing transaction records securely.
- Admin Interface: Web-based admin panel for fraud monitoring and report generation.
- Merchant Interface: Web/mobile dashboard for transaction tracking.

4.2 Communication Interfaces

- Web Interface: Transactions and dashboards accessed via HTTPS protocols.
- Data Exchange: REST APIs using JSON/XML for bank and merchant communication.

5. Performance Requirements

5.1 System Response Time

- Requirement: Authorization responses within 2–3 seconds.
- Requirement: Settlement batch processing completed within standard banking timelines (e.g., T+1).

5.2 Scalability

- Requirement: System should handle 5000 concurrent transactions during peak loads.

5.3 Database Performance

- Requirement: Quick query execution for transaction lookup, fraud checks, and reporting.

6. Design Constraints

6.1 Technology Constraints

- Requirement: The system must be developed using secure, stable frameworks such as Java (Spring Boot) or Python (Django/Flask).
- Limitation: Must comply with PCI-DSS standards for credit card data storage and transmission.

6.2 Hardware Constraints

- Requirement: Operate on secure servers with encryption modules (HSMs), at least 16GB RAM, and 2TB storage.

7. Non-Functional Attributes

7.1 Security

- Requirement: All sensitive data (card numbers, CVV) must be encrypted (AES-256).
- Requirement: Use SSL/TLS for all communications.

7.2 Reliability

- Requirement: System must ensure 99.9% uptime.

7.3 Scalability

- Requirement: Must support expansion to handle international transactions.

7.4 Usability

- Requirement: Easy-to-use dashboards for merchants and admins.

8. Preliminary Schedule and Budget

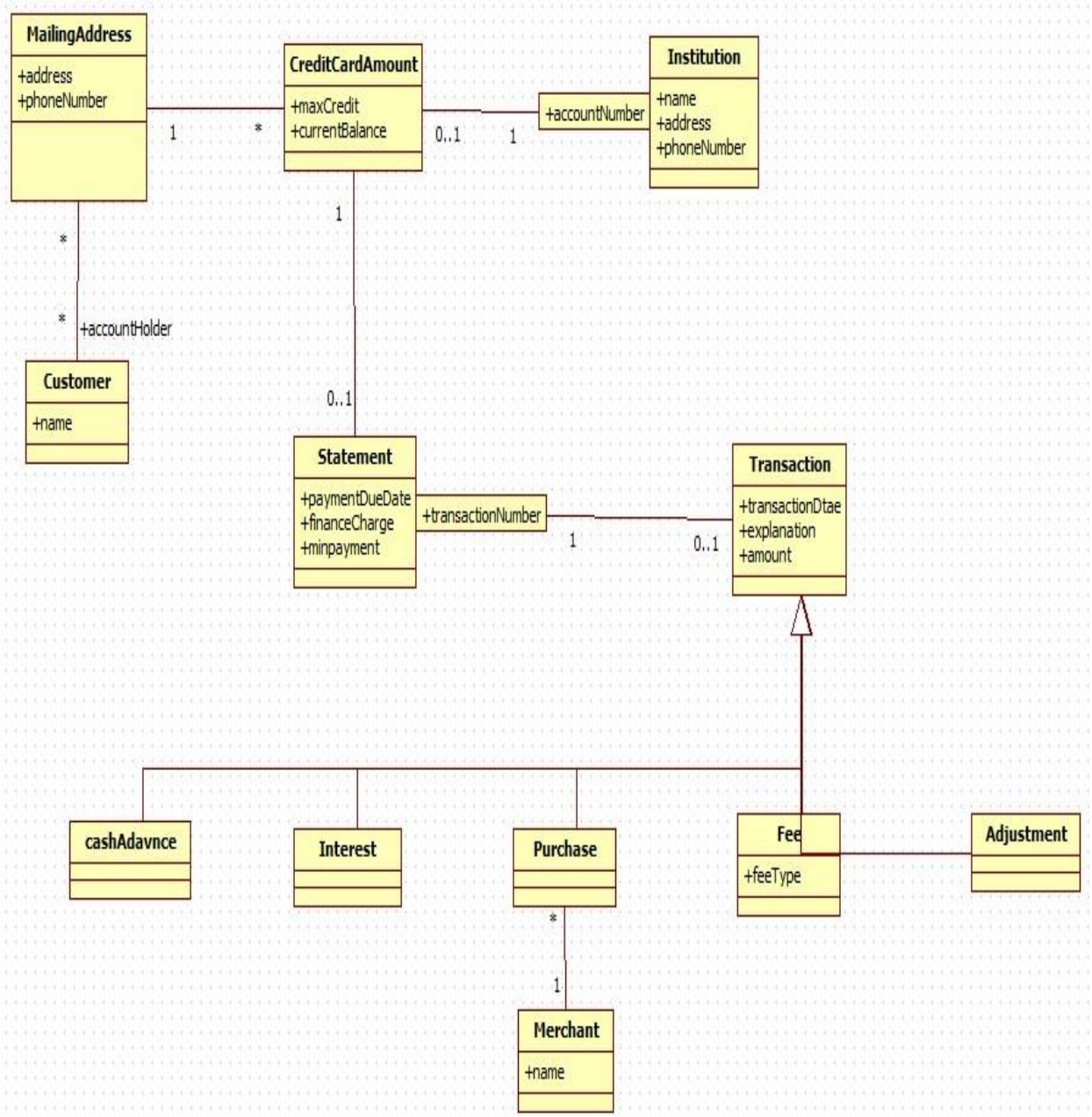
8.1 Development Schedule

- Phase 1 (Design): 2 months
- Phase 2 (Implementation): 4 months
- Phase 3 (Testing): 2 months
- Phase 4 (Deployment and Support): 2 months

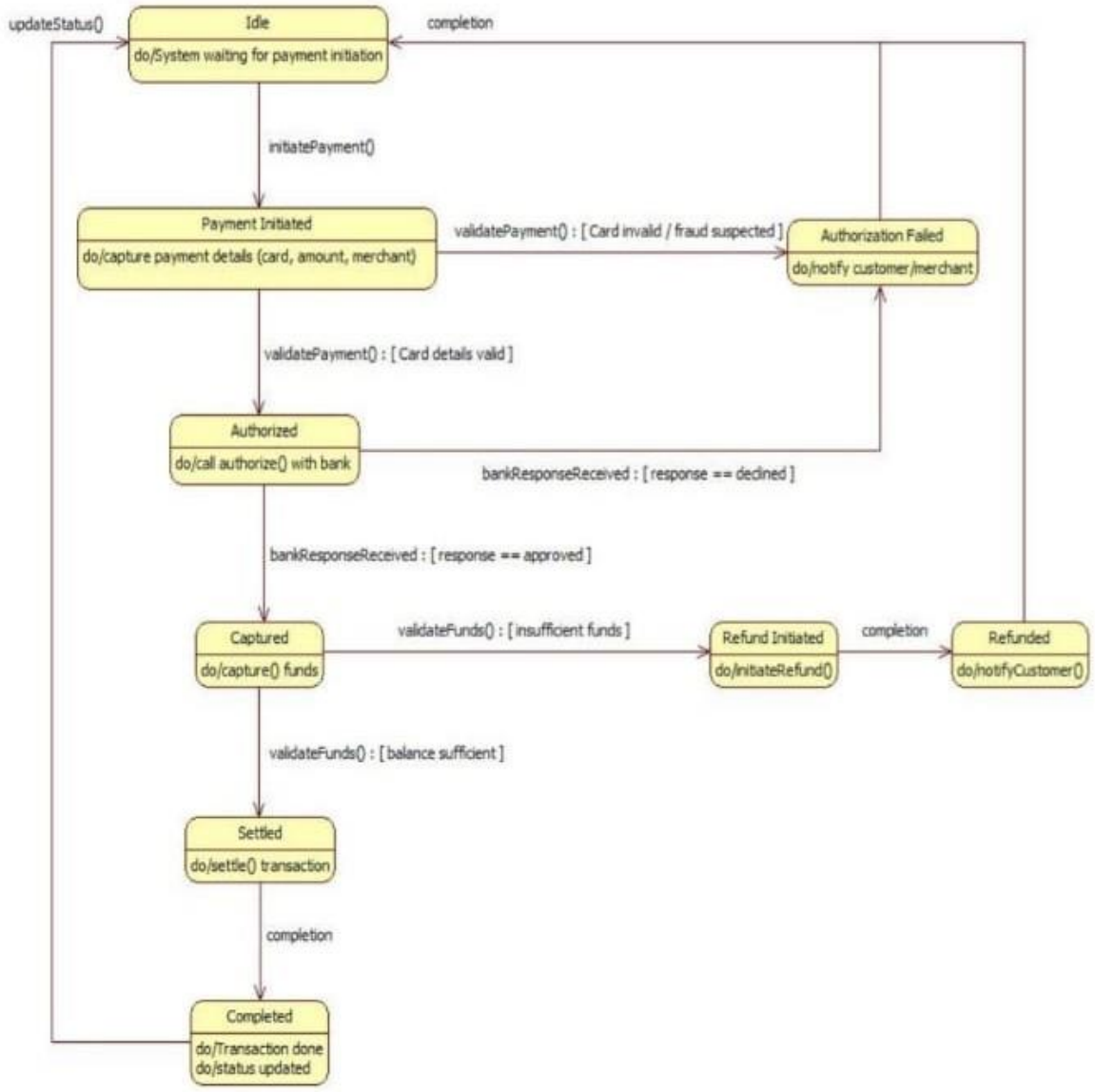
8.2 Budget

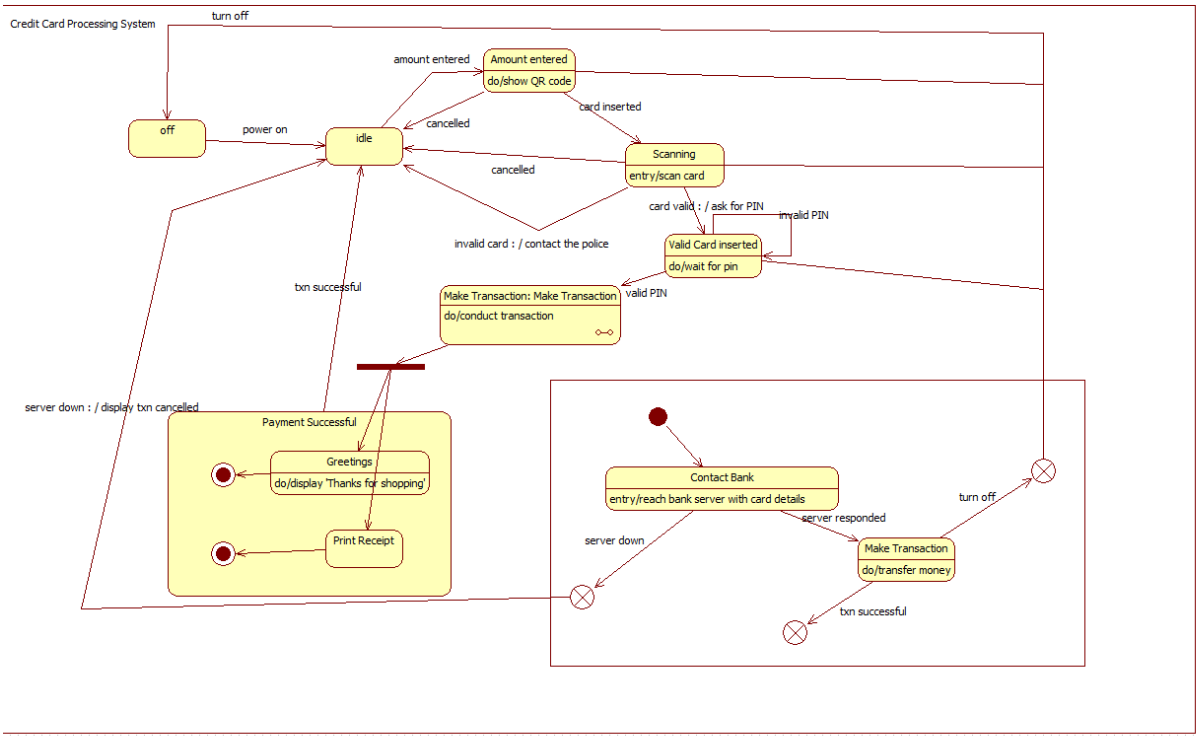
The estimated budget for the development of the Credit Card Processing System is $500,000. This includes costs for development, security certification, testing, deployment, and maintenance.

Class Diagram

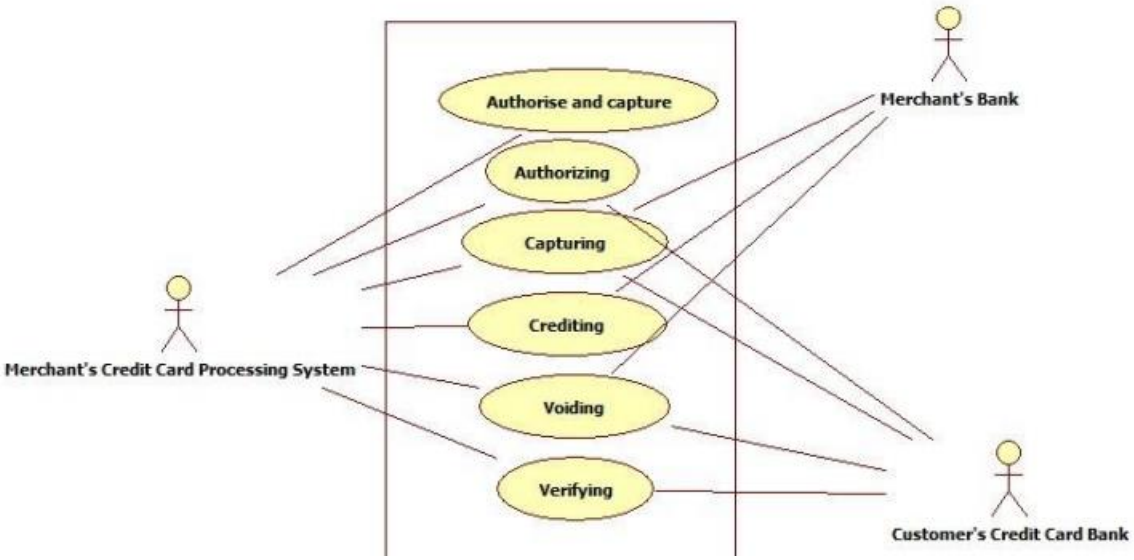


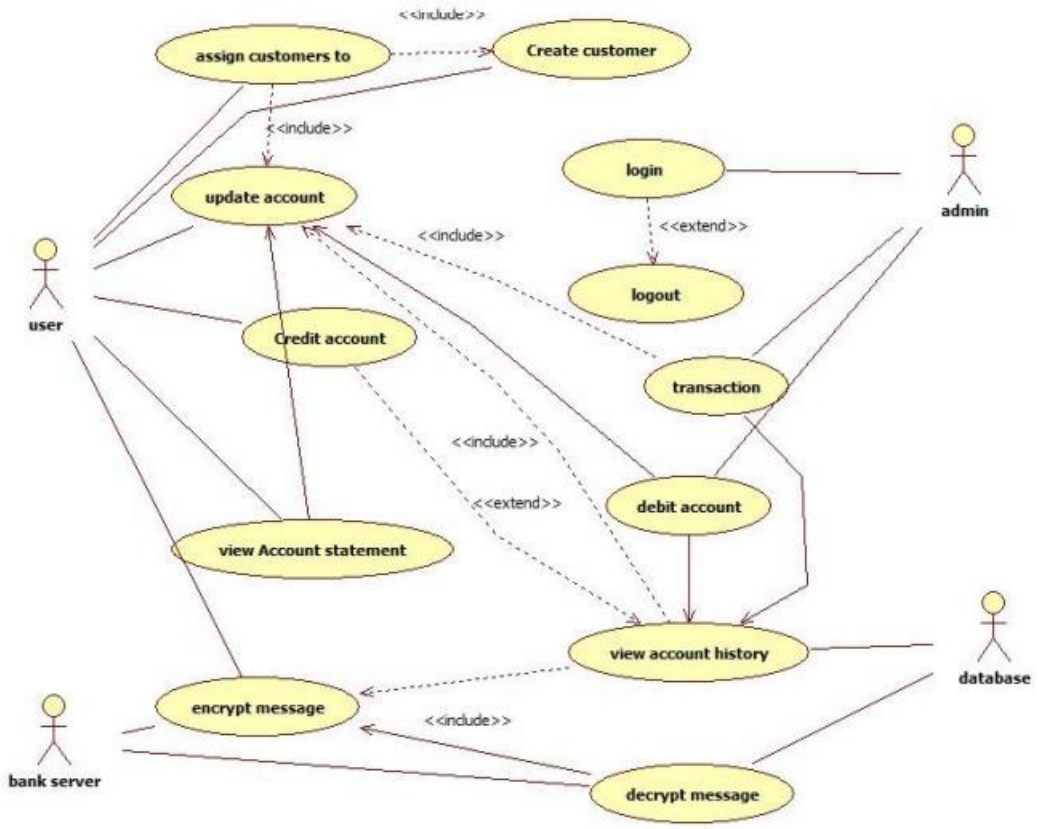
State Diagram



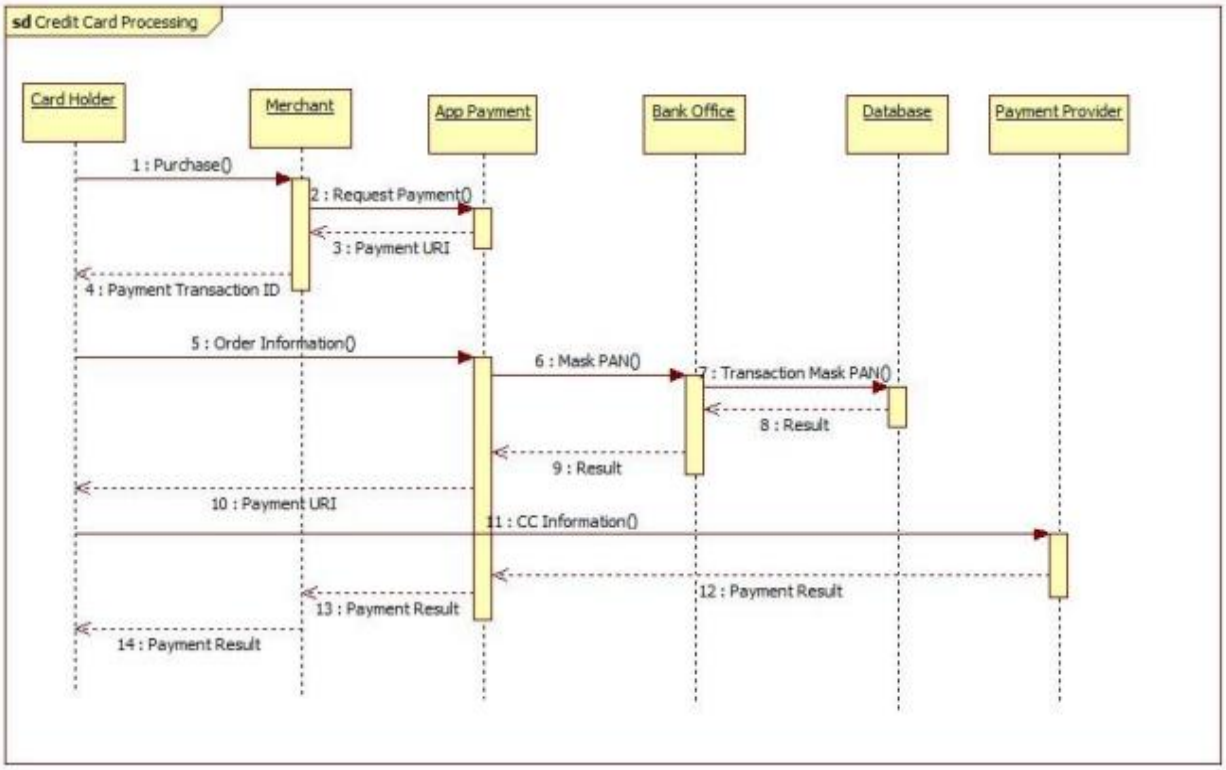


Use case Diagram

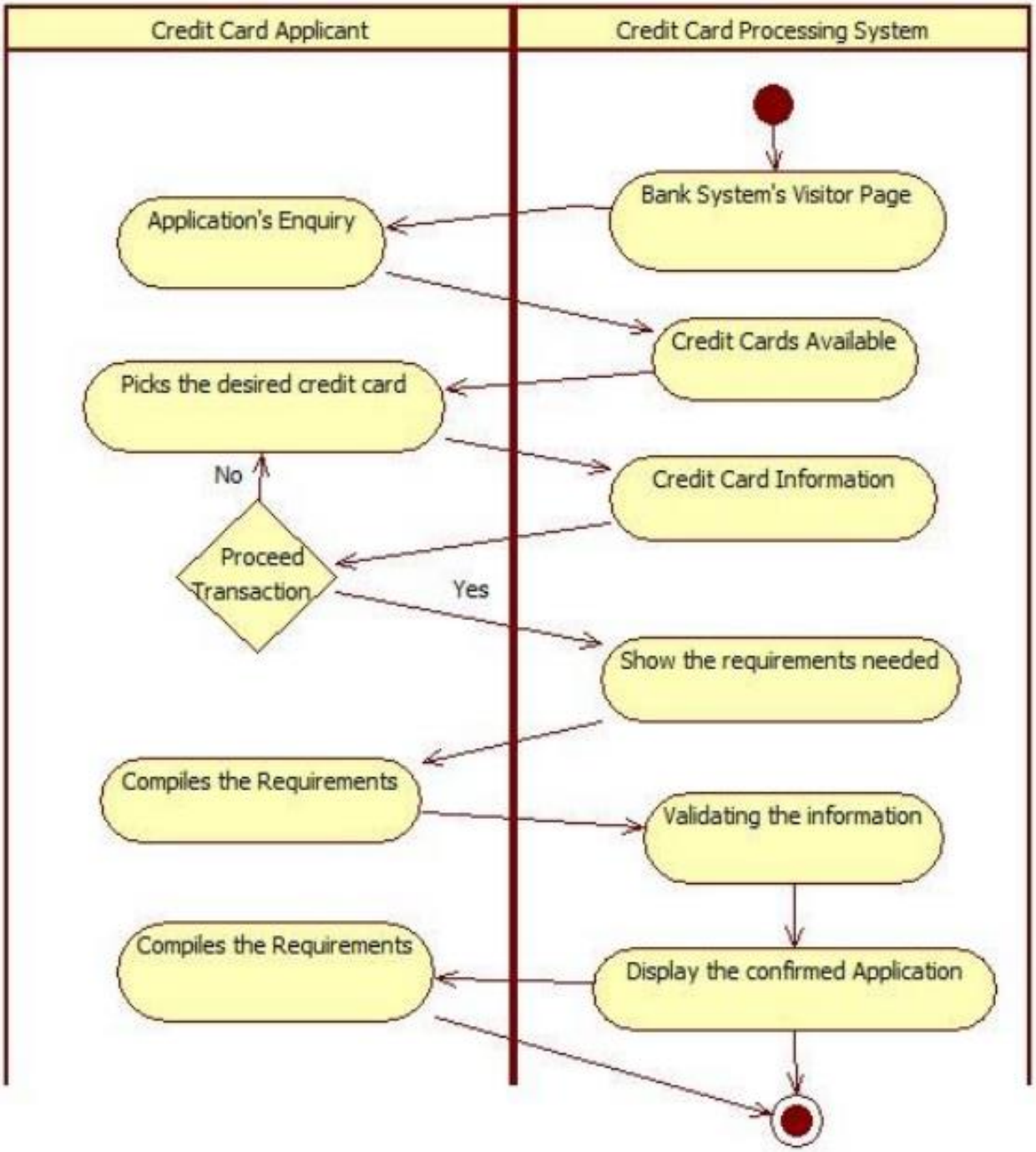




Sequence Diagram



Activity Diagram



3. Library Management System

Problem Statement

Libraries often struggle with keeping track of books and users. If things are done manually, books get misplaced, fines are hard to calculate, and searching for a book takes a lot of time. Students and librarians both face difficulties. A system is needed to make borrowing, returning, and managing books simple and fast.

SRS

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a clear, concise, and detailed Software Requirements Specification (SRS) for the Library Management System (LMS). This document defines the functionality, features, constraints, and the general system requirements for the LMS to ensure the successful development and implementation of the system.

1.2 Scope of this Document

This document covers all the essential information related to the development of the Library Management System, including functional and non-functional requirements, user interfaces, design constraints, and other system attributes. The system will be designed to automate the process of managing library resources, such as books, journals, and memberships. The system will handle the management of books, member registration, book issuing, return, overdue fines, and generating reports.

1.3 Overview

The Library Management System (LMS) is designed to automate the processes involved in managing library resources. It enables users (members) to browse, borrow, and return books, while administrators and librarians can manage books, members, and issue books. The LMS will also include an interface for generating reports, handling overdue fines, and maintaining accurate records of all transactions. The product will include:

Web-based User Interface: Accessible by library members, librarians, and administrators.

Backend System: A database for storing book and member details, along with transaction records.

Admin Panel: For library management and administrative functions.

2. General Description

2.1 General Functions

The main function of the Library Management System is to provide an automated solution to library operations. The system will: Allow members to register, search for books, borrow, and return them. Enable librarians to manage the issuing, returning, and availability status of books. Facilitate the management of library members (registration, modification, deletion). Enable administrators to generate reports, handle overdue fines, and maintain system records.

2.2 User Characteristics

The Library Management System will have three distinct types of users:

Admin: The admin has full control over the system, including managing books, users, fines, and reports.

Librarian: The librarian will be responsible for issuing and receiving books, managing due dates, and handling fines.

Member: The member can search for books, borrow books, check due dates, and pay fines.

3. Functional Requirements

3.1 User Registration and Authentication

Requirement: Users must be able to register and log into the system.

Outcome: Users will be able to access personalized services (e.g., borrow books, check fines, renew books).

Details: The system must authenticate users using secure login credentials (email/username and password).

3.2 Book Management

- Requirement: The system must allow admins and librarians to manage books (add, update, delete).
- Outcome Library books will be correctly managed and updated in real-time.
- Details: Admins can add books with detailed information (title, author, ISBN, genre, etc.) and can track the status of each book (available, issued, overdue).

3.3 Borrowing and Returning Books

- Requirement: The system must track the borrowing and returning of books.
- Outcome: The library will have an up-to-date record of which books are currently issued and their due dates.
- Details: When a book is borrowed, the system should update its status to "issued" and set a due date. Upon return, the book status is updated to "available".

3.4 Fine Management

- Requirement: The system must calculate and track fines for overdue books.
- Outcome: Members who return books late will be fined according to the library’s fine policy.
- Details Fines should be calculated based on the number of overdue days, and the system should notify the user of outstanding fines.

4. Interface Requirements

4.1 Software Interfaces

- Database: The system must interface with a relational database to store and retrieve data related to books, members, and transactions.

- Admin Interface: Admins will interact with the system via a web-based interface to manage users, books, and generate reports.
- Member Interface: Members will interact with the system via a web interface to search for books, borrow books, check fines, and manage their accounts.

4.2 Communication Interfaces

- Web Interface: The software should communicate through HTTP/HTTPS protocols.
- Data Exchange: Data exchange between the frontend and backend should be done using REST APIs (JSON format).

5. Performance Requirements

5.1 System Response Time

- Requirement: The system should return search results for books within 2 seconds.
- Requirement: Actions such as logging in, borrowing books, and returning books should be completed within 5 seconds.

5.2 Scalability

- Requirement: The system should be able to handle up to 1000 concurrent users without degradation in performance.

5.3 Database Performance

- Requirement: The database should support quick query execution for tasks like book search, user lookup, and transaction recording.

6. Design Constraints

6.1 Technology Constraints

- Requirement: The system must be developed using the latest stable versions of web technologies like HTML5, CSS3, JavaScript, and Python (Django) or Java (Spring Boot) for the backend.

- Limitation: The system should be compatible with modern web browsers (Chrome, Firefox, Edge).

6.2 Hardware Constraints

- Requirement: The system must operate on standard hardware, which includes servers with at least 8GB of RAM and 1TB of storage for initial deployment.

7. Non-Functional Attributes

7.1 Security

- Requirement: User data, including login credentials, must be encrypted using modern encryption techniques (e.g., AES, bcrypt).
- Requirement The system must prevent unauthorized access and support role-based access control (RBAC).

7.2 Reliability

- Requirement: The system should have 99.9% uptime, ensuring availability during peak library usage times.

7.3 Scalability

- Requirement: The system should support scaling to handle increased usage, with the ability to add more servers as needed.

8. Preliminary Schedule and Budget

8.1 Development Schedule

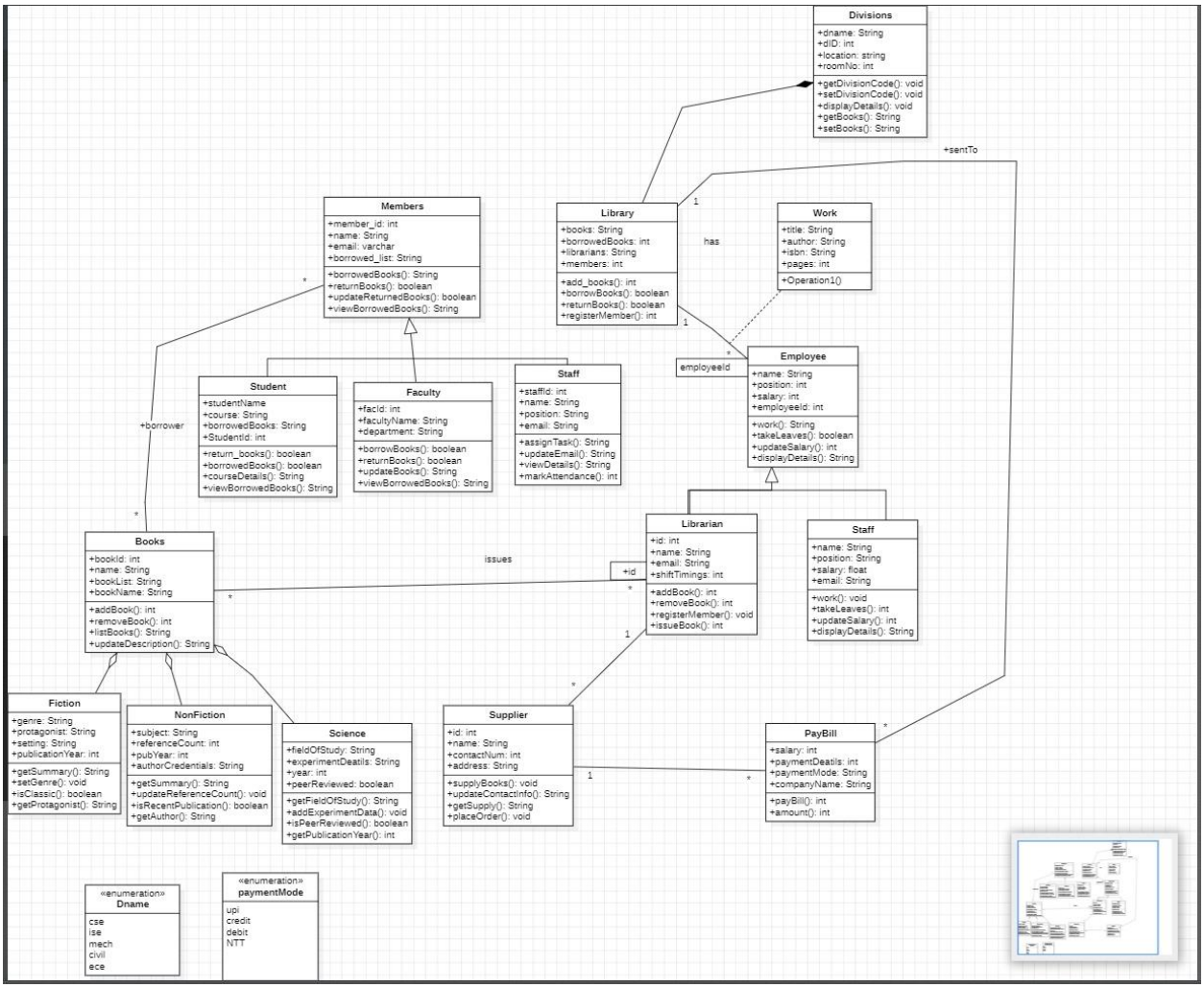
- Phase 1 (Design): 1 month
- Phase 2 (Implementation): 3 months
- Phase 3 (Testing): 1 month

- Phase 4 (Deployment and Support): 1 month

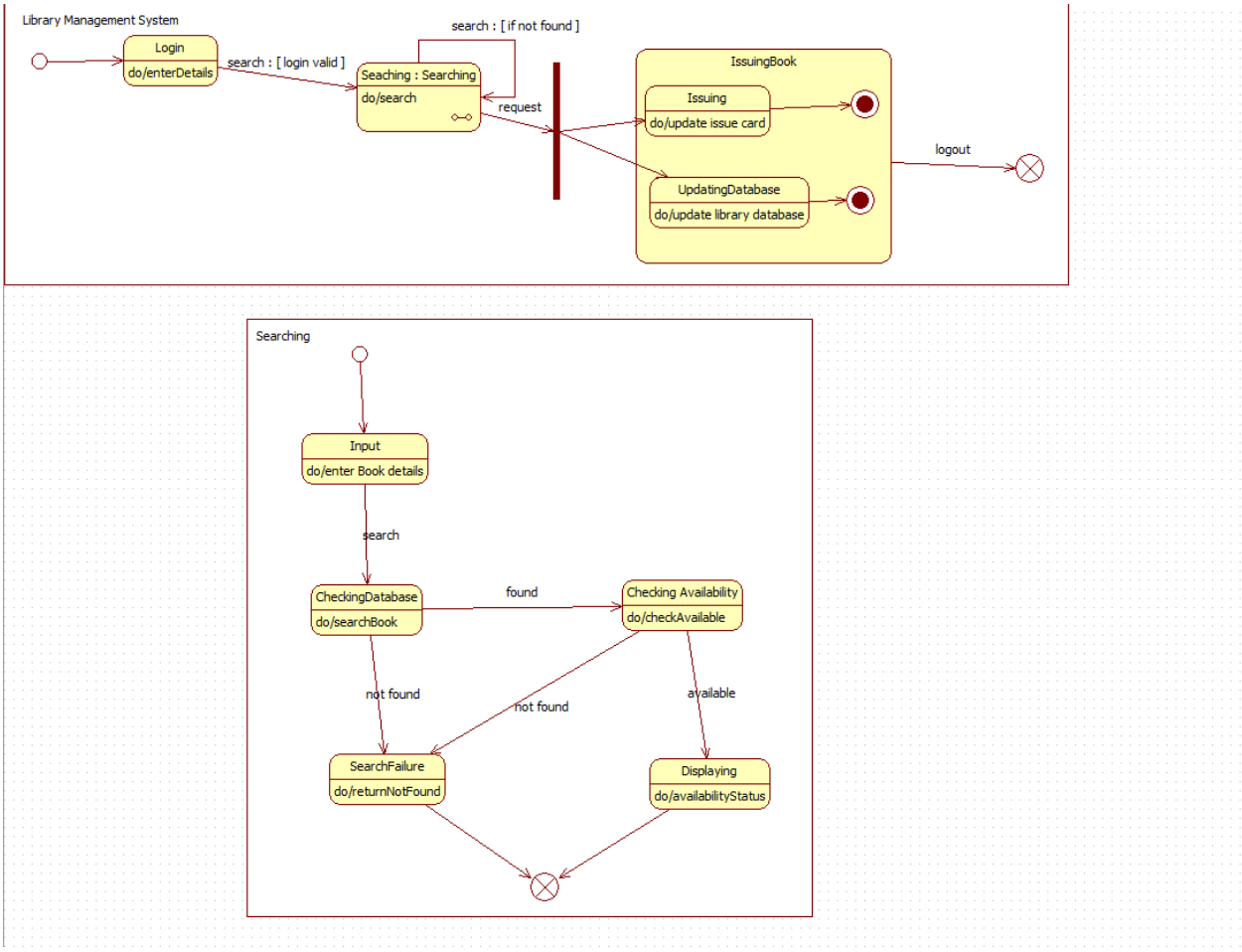
8.2 Budget

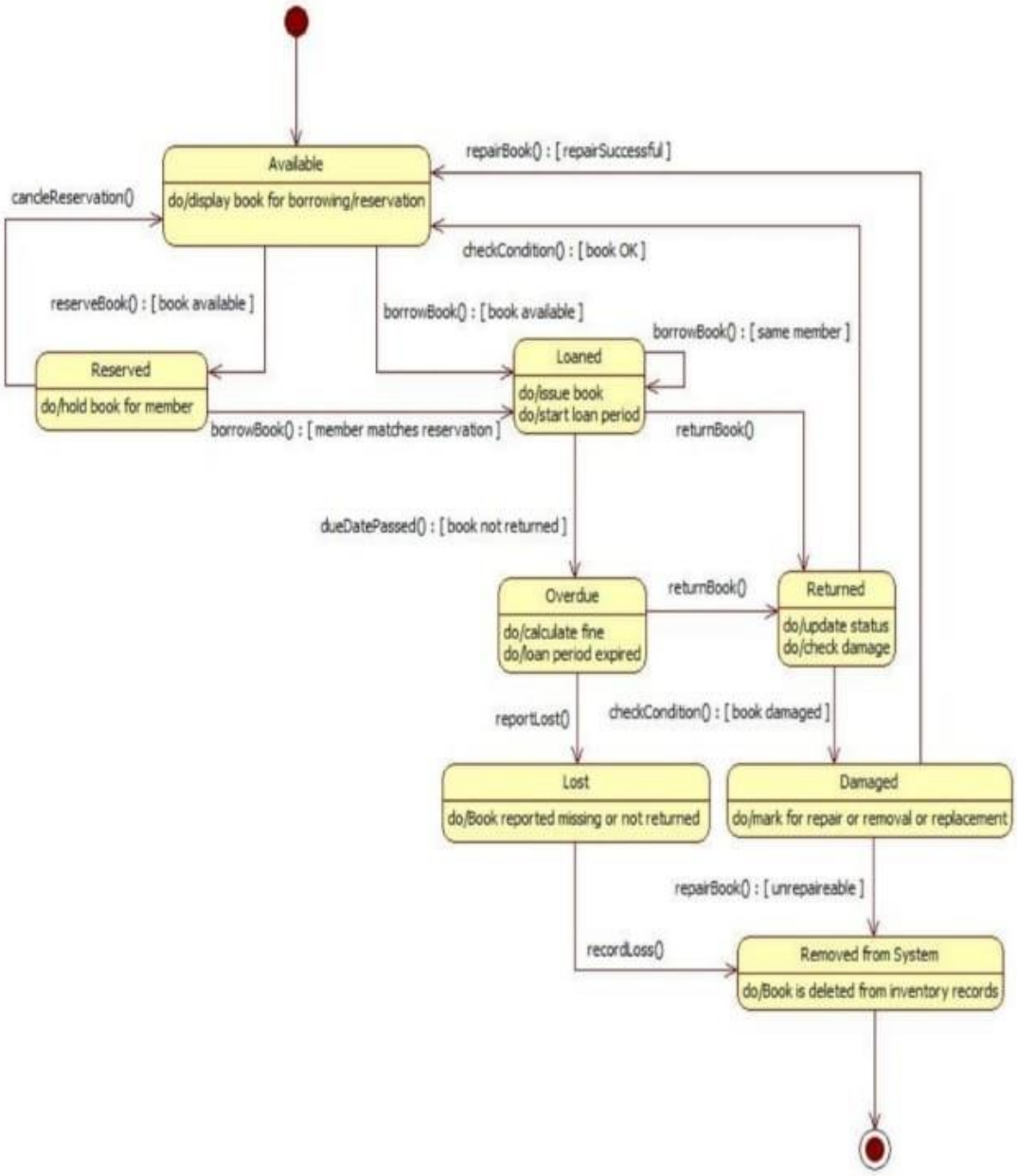
The estimated budget for the development of the Library Management System is $50,000. This includes costs for development, testing, deployment, and initial support kindly give me SRS document same as this for credit card processing system

Class Diagram

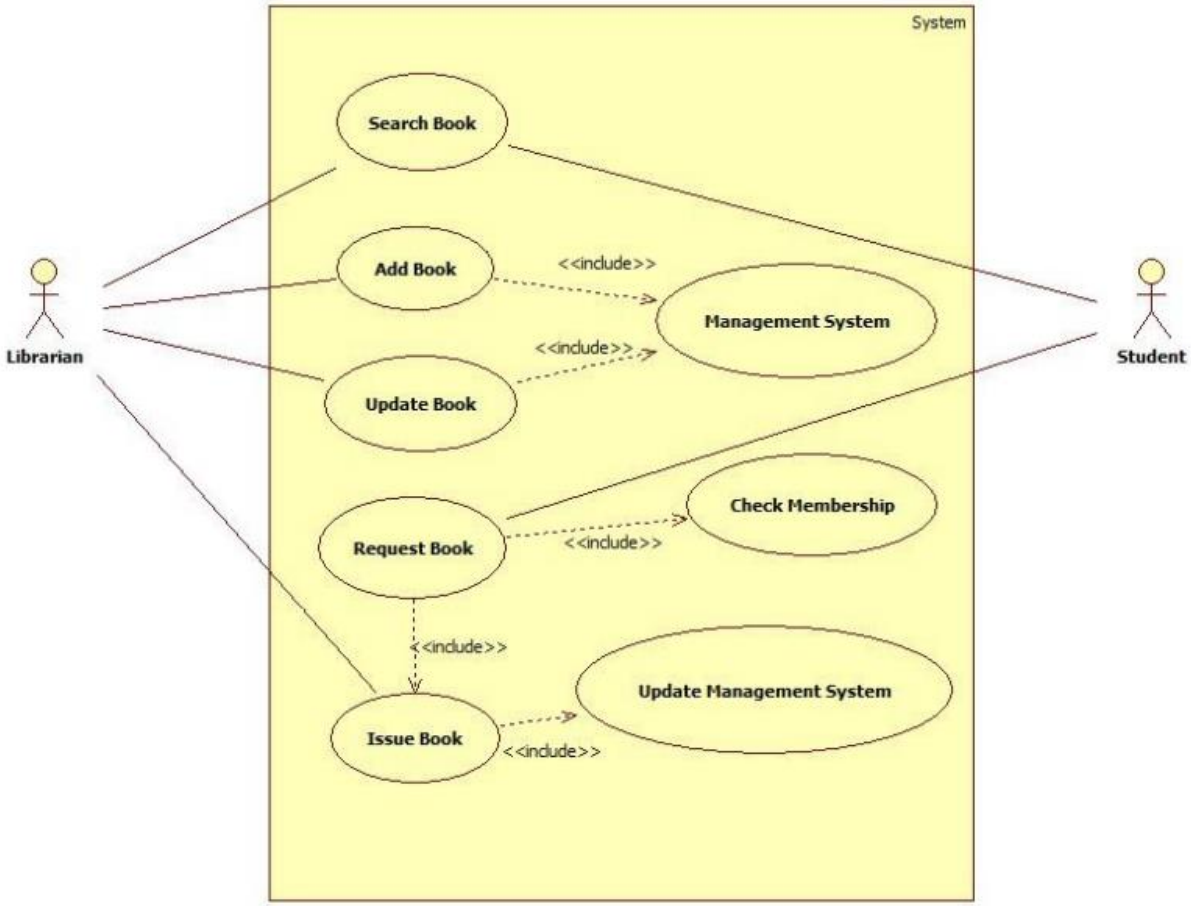


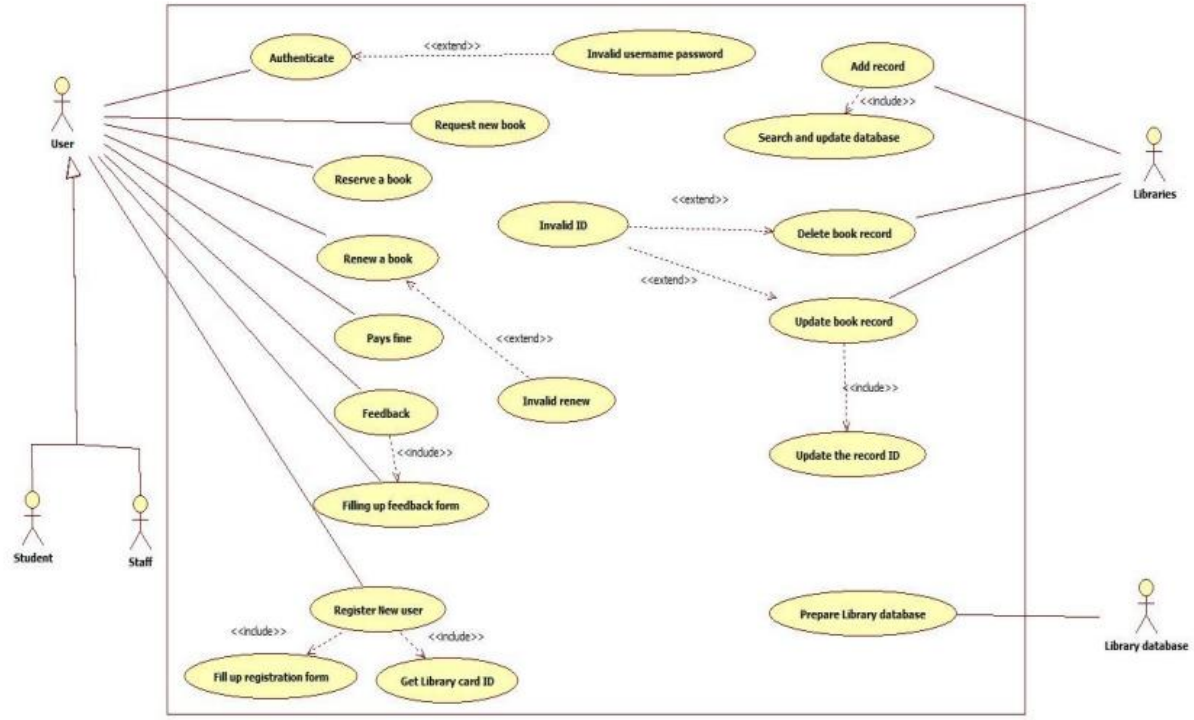
State Diagram



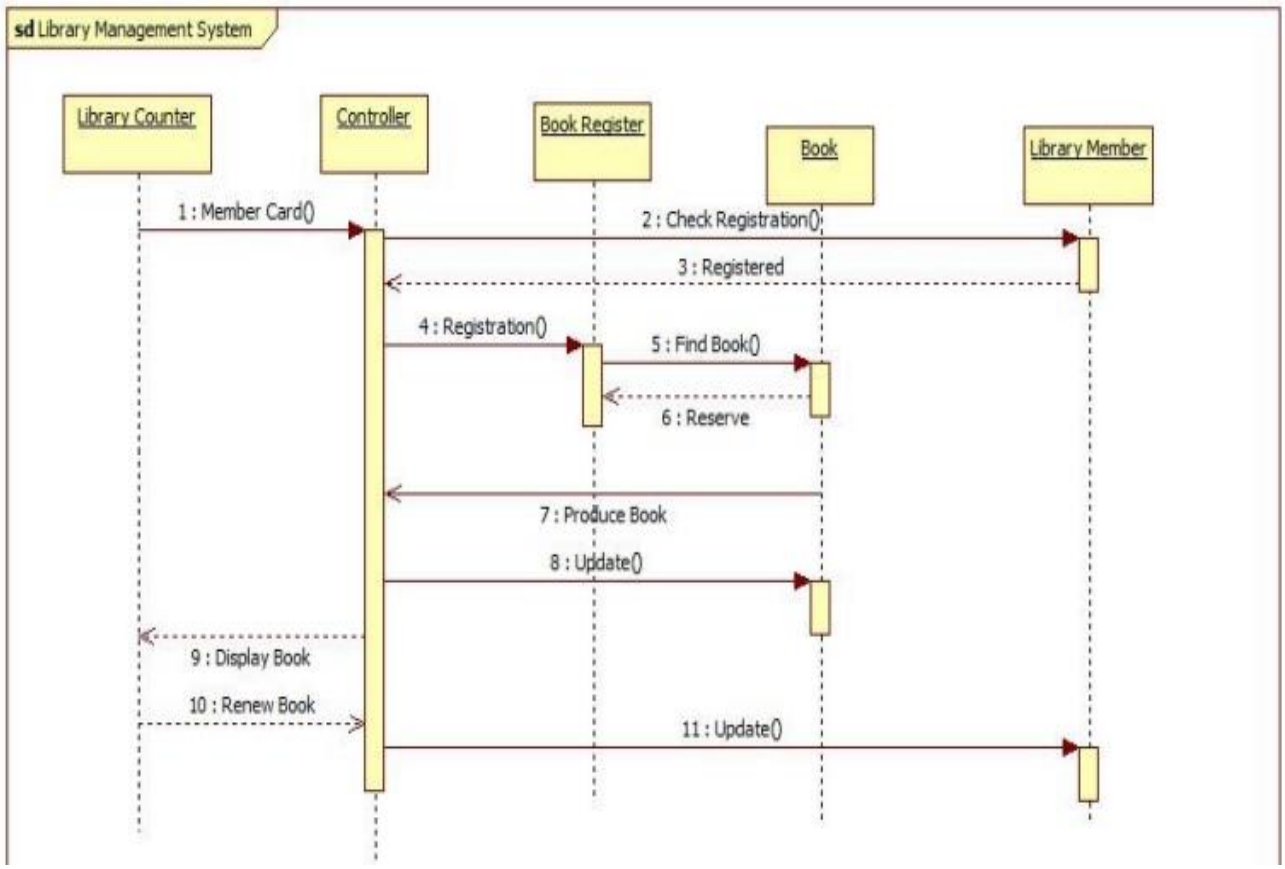

Available
do/display book for borrowing/reservation
repairBook() : [repairSuccessful]
cancleReservation()
checkCondition() : [book OK]
reserveBook() : [book available]
borrowBook() : [book available]
borrowBook() : [same member]
Reserved
do/hold book for member
Loaned
do/issue book
do/start loan period
borrowBook() : [member matches reservation]
returnBook()
dueDatePassed() : [book not returned]
Overdue
do/calculate fine
do/loan period expired
returnBook()
Returned
do/update status
do/check damage
reportLost()
checkCondition() : [book damaged]
Lost
do/Book reported missing or not returned
Damaged
do/mark for repair or removal or replacement
repairBook() : [unrepaireable]
recordLoss()
Removed from System
do/Book is deleted from inventory records

Use case Diagram

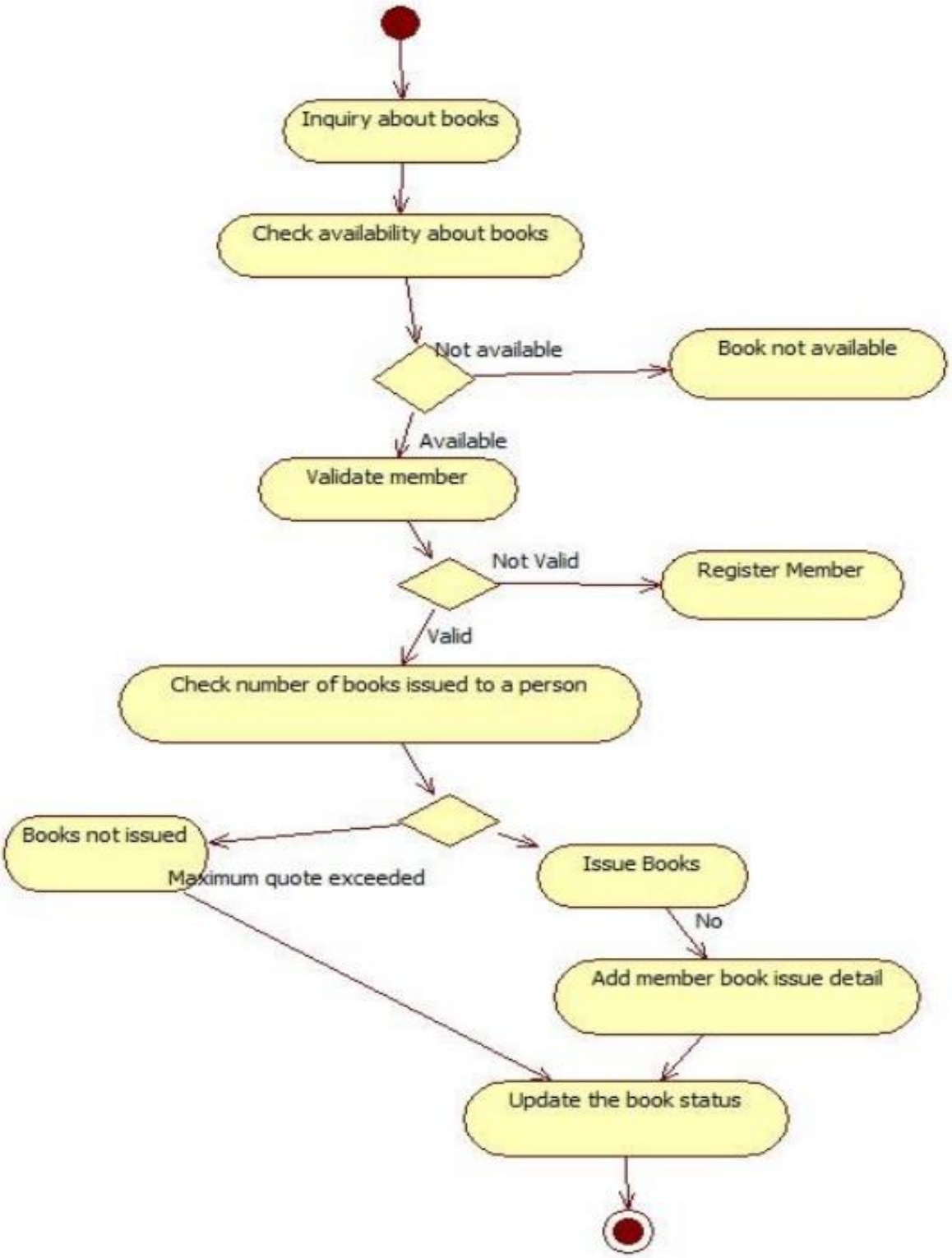




Sequence Diagram



Activity Diagram



4. Stock Maintenance

Problem Statement

Shops and warehouses need to know how much stock they have at any moment. If stock is managed manually, it can lead to missing items, overstocking, or expired goods. This causes losses and customer dissatisfaction. A system is needed that can update stock levels automatically and give real-time alerts.

SRS

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a clear and detailed Software Requirements Specification (SRS) for the Stock Maintenance System (SMS). It defines the functionality, features, constraints, and general requirements needed for the successful development and deployment of the system.

1.2 Scope of this Document

This document covers functional and non-functional requirements, user interfaces, performance constraints, and design considerations for the SMS. The system will automate stock tracking, purchase management, sales updates, inventory alerts, and reporting for shops, warehouses, and businesses.

1.3 Overview

The SMS is designed to help businesses track and manage stock in real time, avoid overstocking or shortages, and generate accurate reports.

The product will include:

- Web-Based User Interface: Accessible by store managers, staff, and administrators.
- Backend Database: Stores stock details, supplier records, sales, and purchase data.
- Admin Panel: For high-level reporting and business analytics.

2. General Description

2.1 General Functions

The system will:

- Track product stock levels and update automatically on sales or purchases.
- Manage supplier details and purchase orders.
- Send alerts for low stock or expired products.
- Provide sales and inventory reports.

2.2 User Characteristics

The system will have three user types:

- Admin: Full control over stock, reports, and system settings.
- Manager: Manages stock levels, suppliers, and reporting.
- Staff: Updates daily sales/purchases and checks stock status.

3. Functional Requirements

3.1 User Authentication

- Requirement: Secure login for all users.
- Outcome: Protects system from unauthorized access.

3. Stock Management

- Requirement: Add, update, and delete product details.
- Outcome: Accurate tracking of available products.

3.3 Sales & Purchase Management

- Requirement: Update stock automatically when sales/purchases occur.
- Outcome: Real-time inventory status.

3.4 Alerts & Notifications

- Requirement: Notify when stock reaches reorder level.
- Outcome: Prevents shortages and delays.

3.5 Report Generation

- Requirement: Generate daily, weekly, or monthly reports.
- Outcome: Better decision-making with accurate data.

4. Interface Requirements

4. Software Interfaces

- Relational database (MySQL/Oracle).
- Admin dashboard for analytics.
- Staff UI for data entry.

4.2 Communication Interfaces

- Web interface over HTTPS.
- Data exchange through REST APIs.

5. Performance Requirements

- Stock updates must reflect in real time (<2 seconds).
- System must handle 2000 concurrent transactions.
- Database queries must respond within 3 seconds.

6. Design Constraints

- Must use modern frameworks (Java Spring Boot / Python Django).
- Must support web and mobile devices.
- Standard hardware: 8GB RAM, 1TB storage.

7. Non-Functional Attributes

- Security: Role-based access, encryption of business data.
- Reliability: 99% uptime.
- Scalability: Expandable for large warehouses.
- Usability: Simple UI for non-technical staff.

8. Preliminary Schedule and Budget

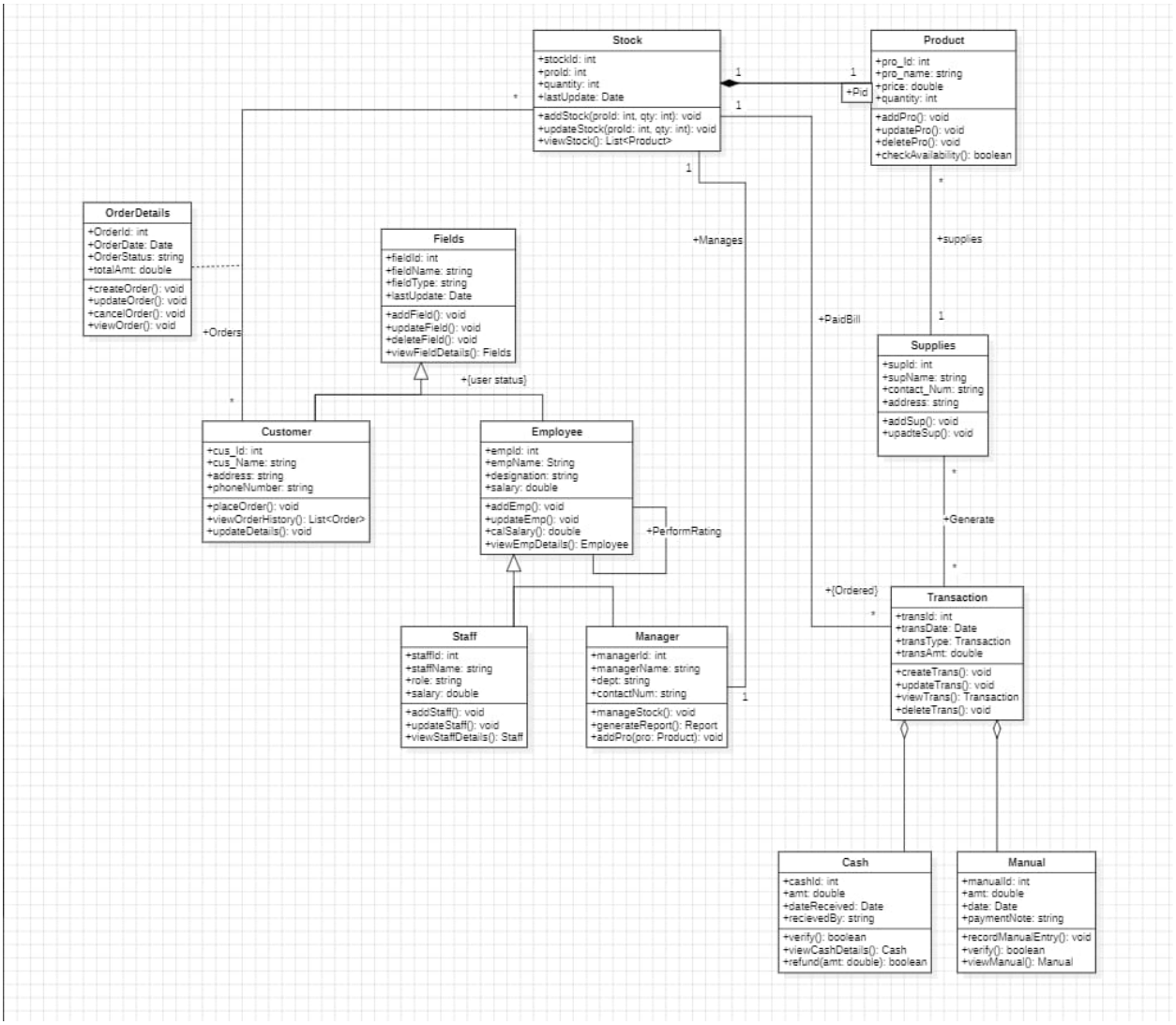
8.1 Development Schedule

- Phase 1 (Design): 1 month
- Phase 2 (Implementation): 2 months
- Phase 3 (Testing): 1 month
- Phase 4 (Deployment & Support): 1 month

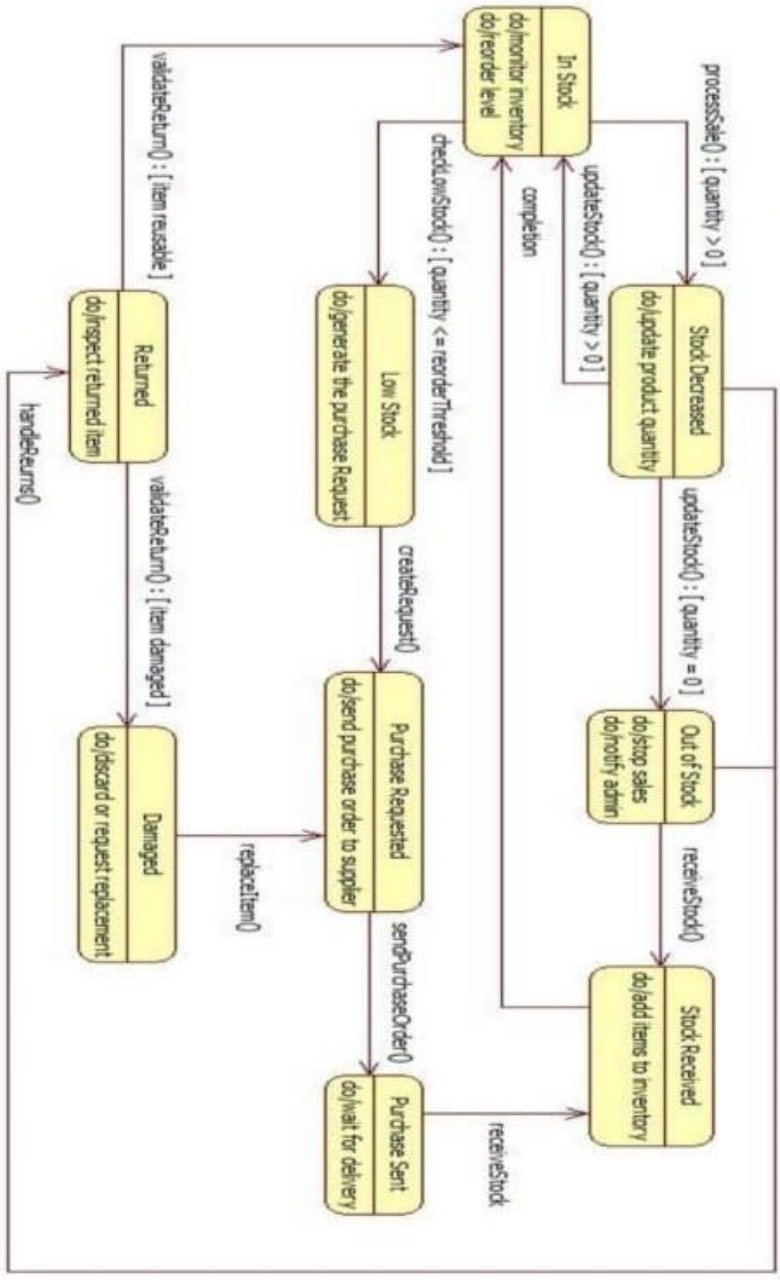
8.2 Budget

The estimated budget for the development of the Stock Maintenance System is $120,000. This includes costs for development, testing, deployment, and initial support kindly give me SRS document same as this for credit card processing system

Class Diagram

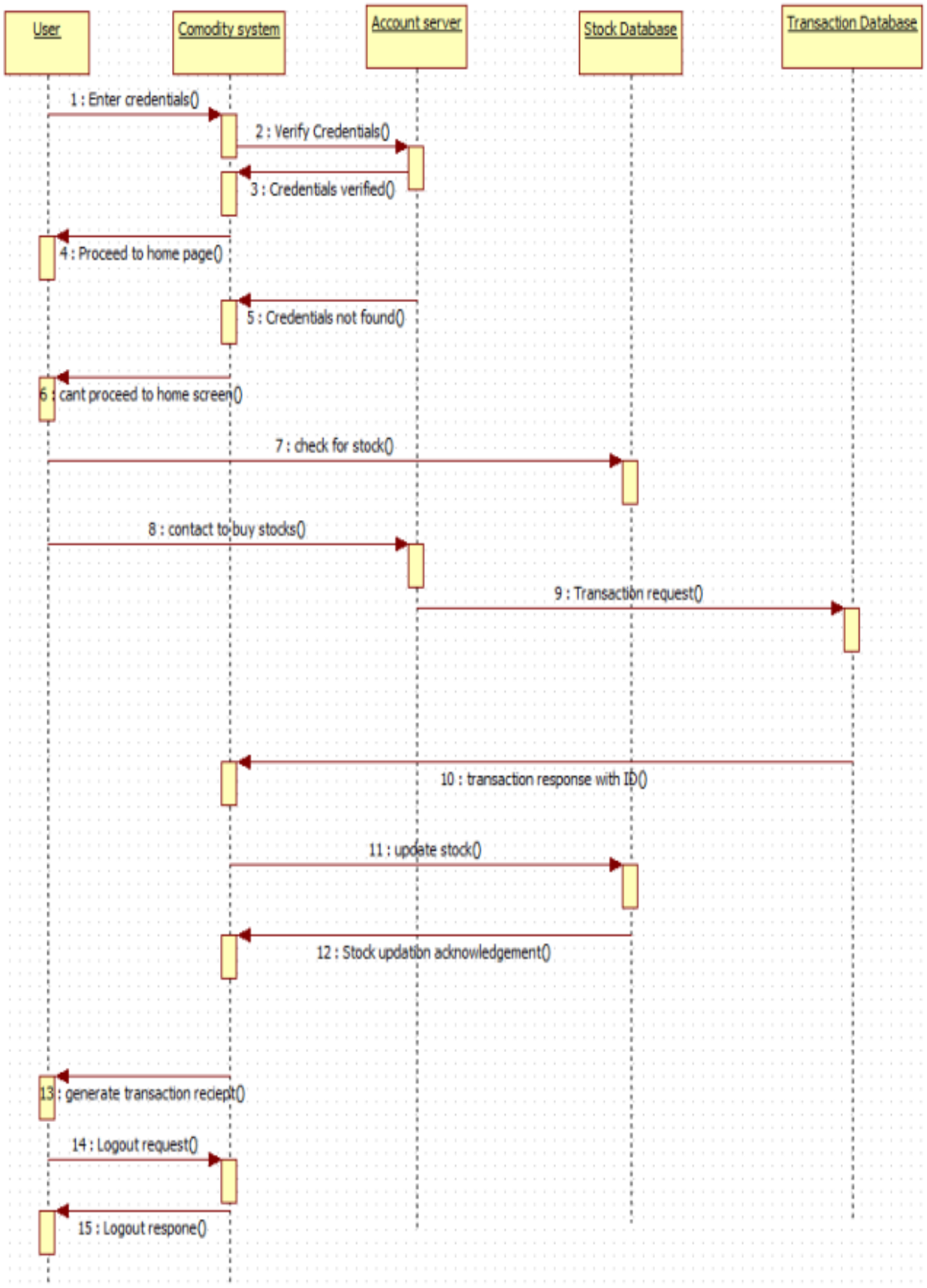


State Diagram



Use case Diagram

Sequence Diagram



Activity Diagram

5.Passport Automation System

Problem statement

Getting a passport often takes too long because of paperwork, queues, and delays in verification. People don’t always know the status of their applications and have to make multiple visits to the office. A system is needed where citizens can apply online, book appointments, and track their passport status easily.

SRS

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a detailed Software Requirements Specification (SRS) for the Passport Automation System (PAS). It outlines the functional, non-functional, and design requirements necessary for successful system development and implementation.

1.2 Scope of this Document

This document defines requirements for automating passport services such as new applications, renewals, appointment scheduling, verification, tracking, and reporting.

1.3 Overview

The PAS is designed to reduce manual work and improve transparency in passport services.

The product will include:

- Web & Mobile Portal: For citizens to apply, track, and renew passports.
- Backend Database: Stores application records, verification details, and status updates.
- Admin Panel: For passport office staff and verification agencies.

2. General Description

2.1 General Functions

The system will:

- Accept online applications and renewals.
- Allow scheduling of appointments.
- Enable integration with police verification and government databases.
- Track status of applications and notify users.
- Generate reports for staff and administrators.

2.2 User Characteristics

- Admin: Controls system operations and reporting.
- Staff: Verifies applications, schedules, and approvals.
- Citizen: Submits applications and tracks status.

3. Functional Requirements

3.1 User Authentication

- Requirement: Secure login for citizens and staff.
- Outcome: Prevents unauthorized access.

3.2 Application Submission

- Requirement: Allow new and renewal applications.
- Outcome: Citizens can complete applications online.

3. Appointment Scheduling

- Requirement: Book and manage appointments.
- Outcome: Organized passport office visits.

3.4 Verification Integration

- Requirement: Integration with police and government ID systems.
- Outcome: Faster and more reliable verification.

3.5 Status Tracking & Notifications

- Requirement: Citizens should track application progress.
- Outcome: Transparency and reduced follow-ups.

4. Interface Requirements

4. Software Interfaces

- Database for storing applications and verification records.
- Admin dashboard for staff and reporting.
- Citizen portal for application and tracking.

4.2 Communication Interfaces

- Secure communication over HTTPS.
- API integration with external government databases.

5. Performance Requirements

- Application submission should take <3 seconds.
- Must support 10,000 concurrent users.
- Database queries should respond within 2 seconds.

6. Design Constraints

- Must comply with government IT and security regulations.
- Must support Aadhaar/ID verification systems.

- Standard hardware: 16GB RAM, 2TB storage.

7. Non-Functional Attributes

- Security: Strong encryption and 2FA for users.
- Reliability: 99.9% uptime.
- Scalability: Support multiple regional passport offices.
- Usability: Clear, step-by-step application process.

8. Preliminary Schedule and Budget

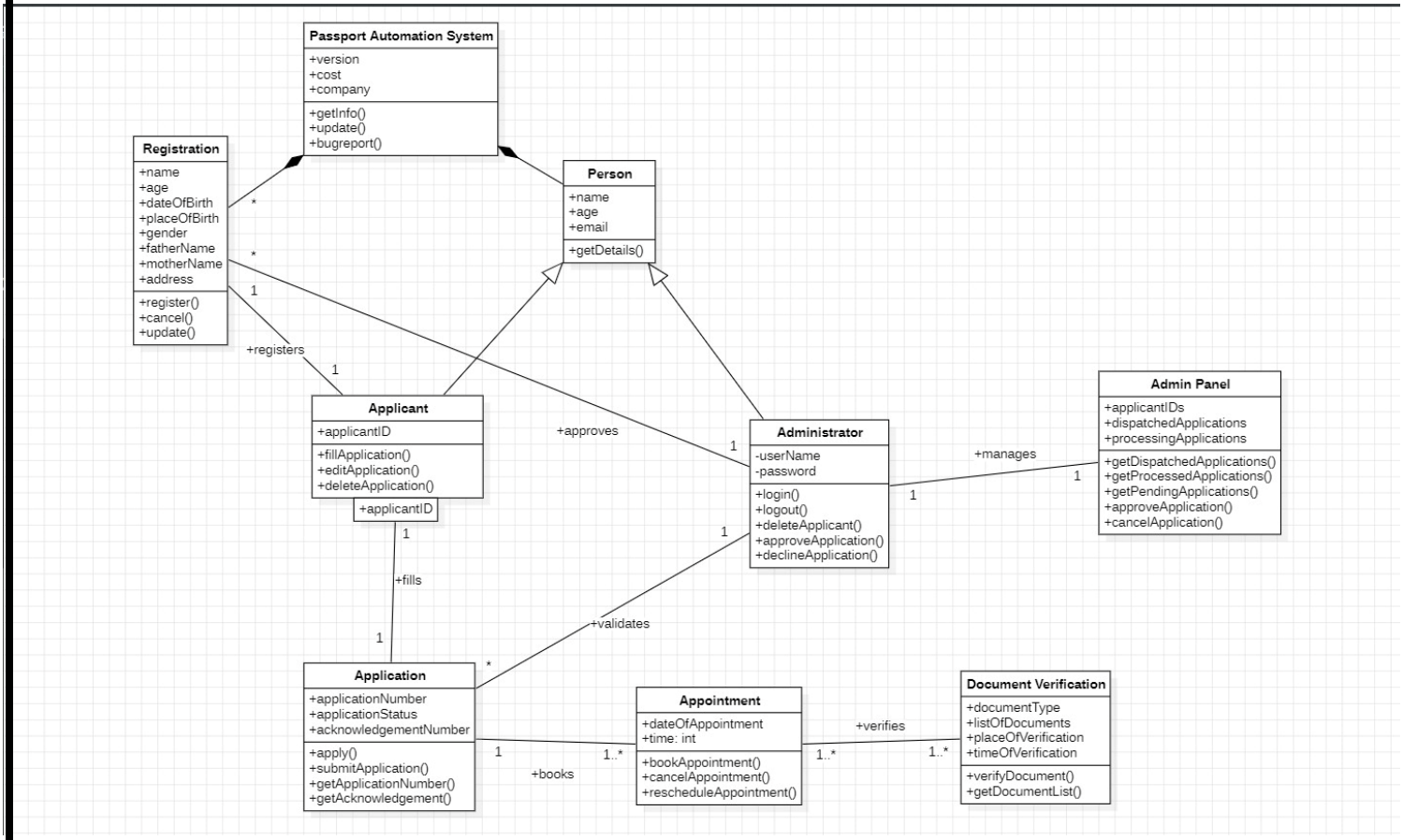
8.1 Development Schedule

- Phase 1 (Design): 2 months
- Phase 2 (Implementation): 4 months
- Phase 3 (Testing): 2 months
- Phase 4 (Deployment & Support): 2 months

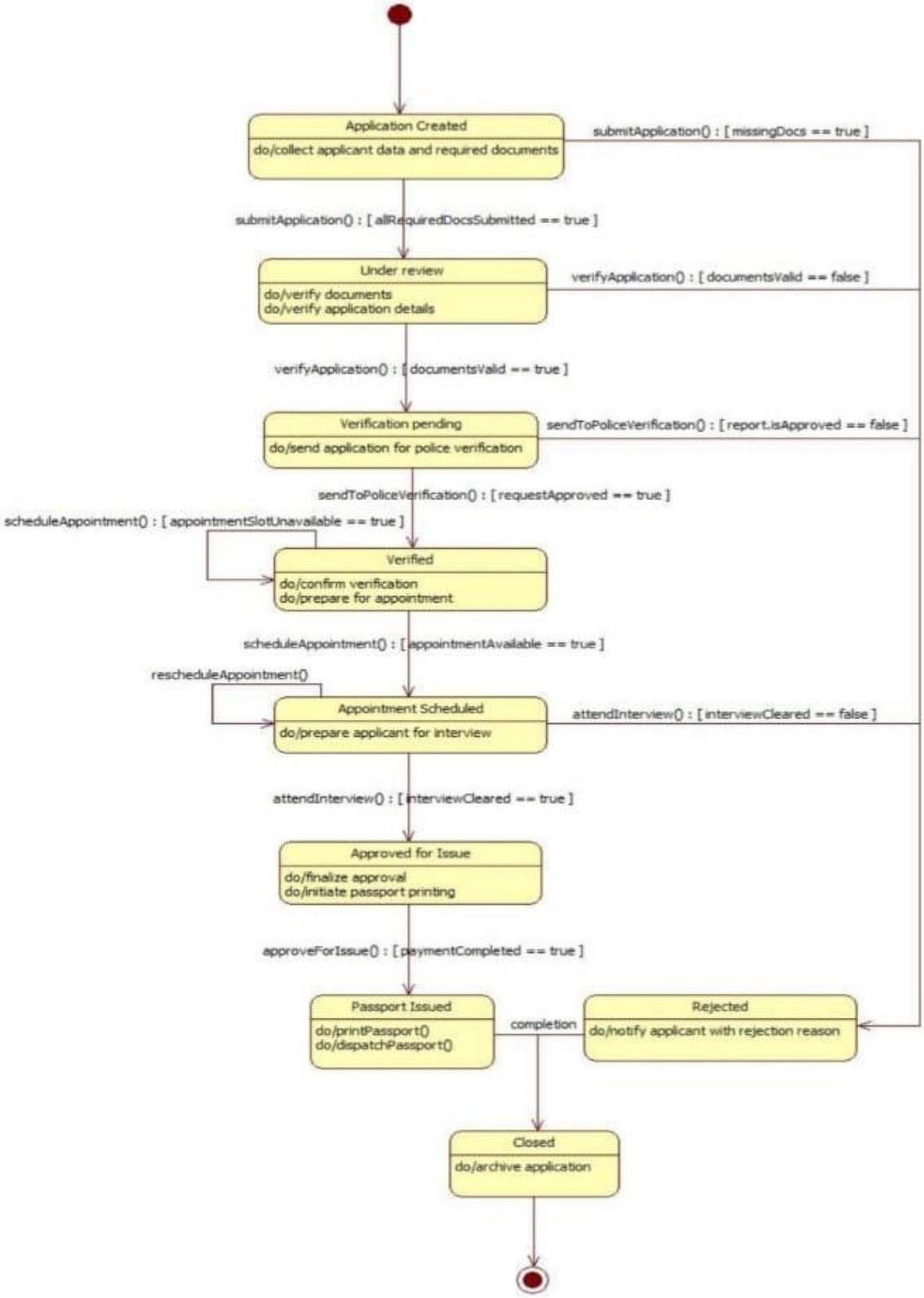
8.2 Budget

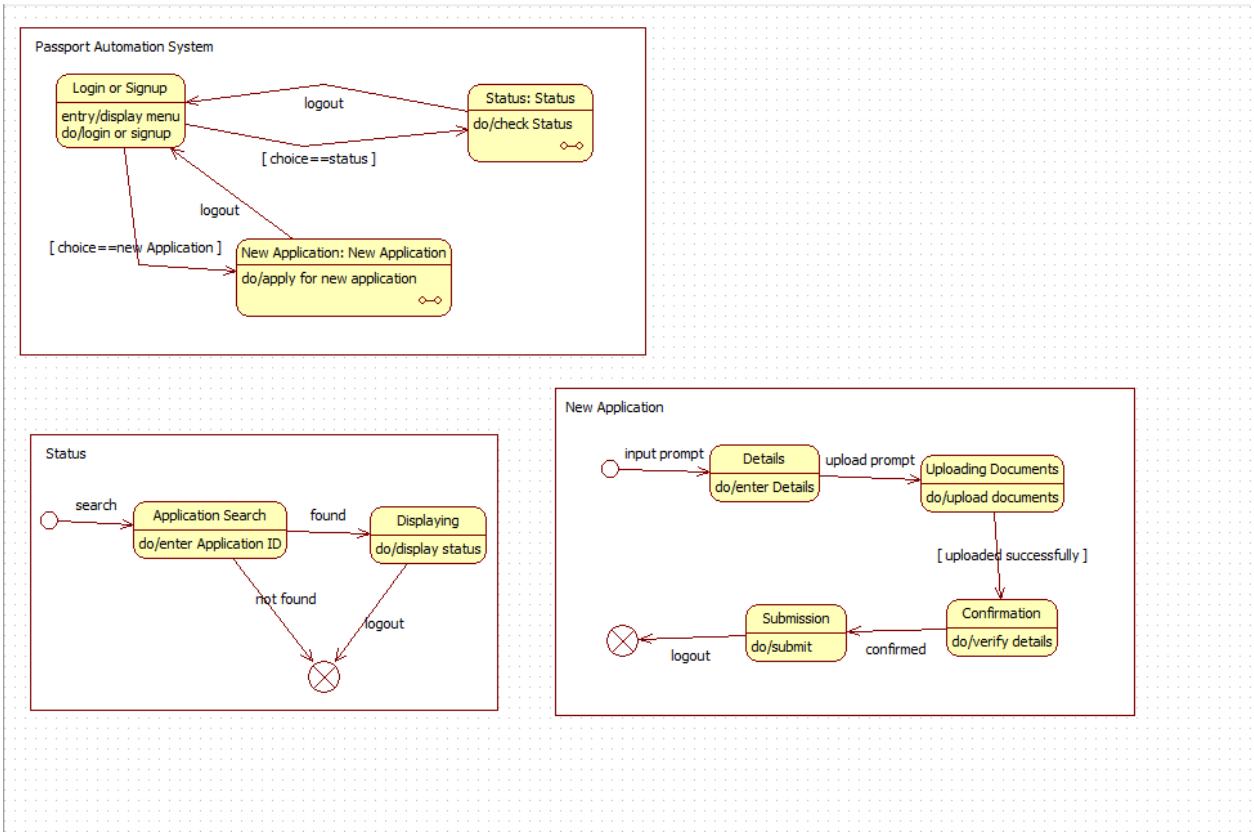
The estimated budget for the development of the Passport Automation Systemis $ 700,000. This includes costs for development, testing, deployment, and initial support kindly give me SRS document same as this for credit card processing system

Class Diagram



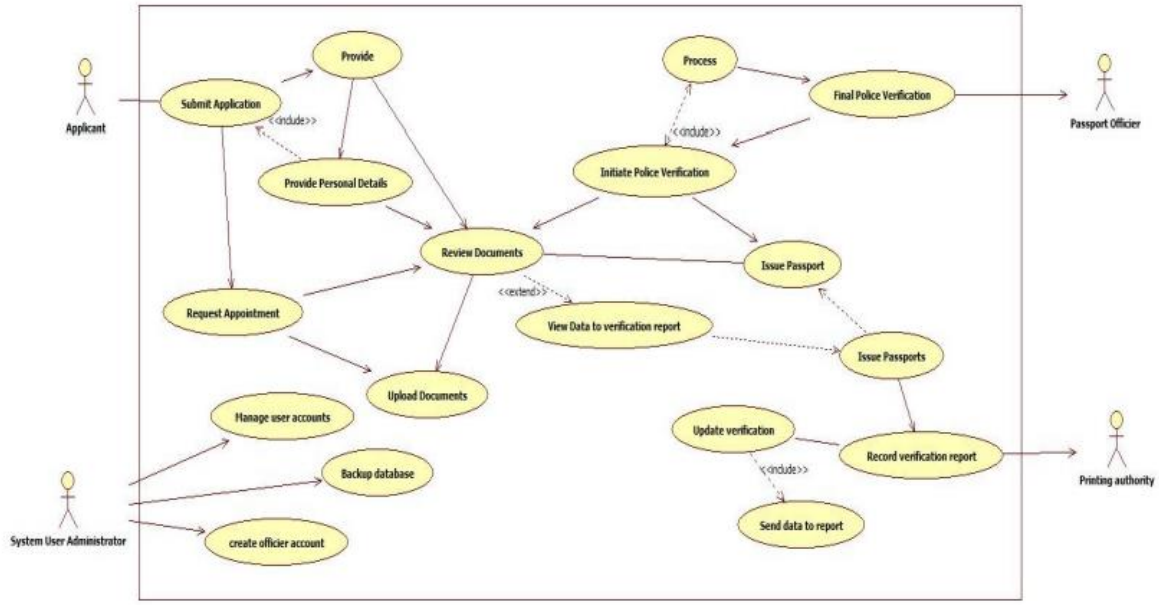
State Diagram

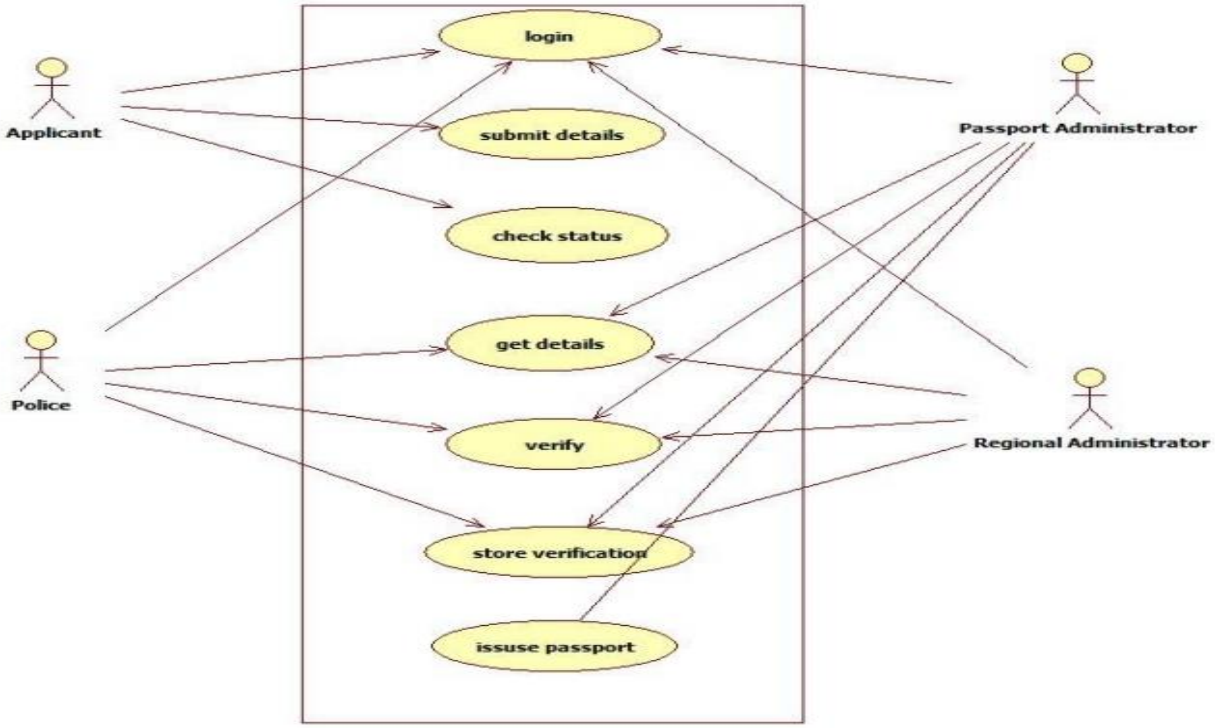




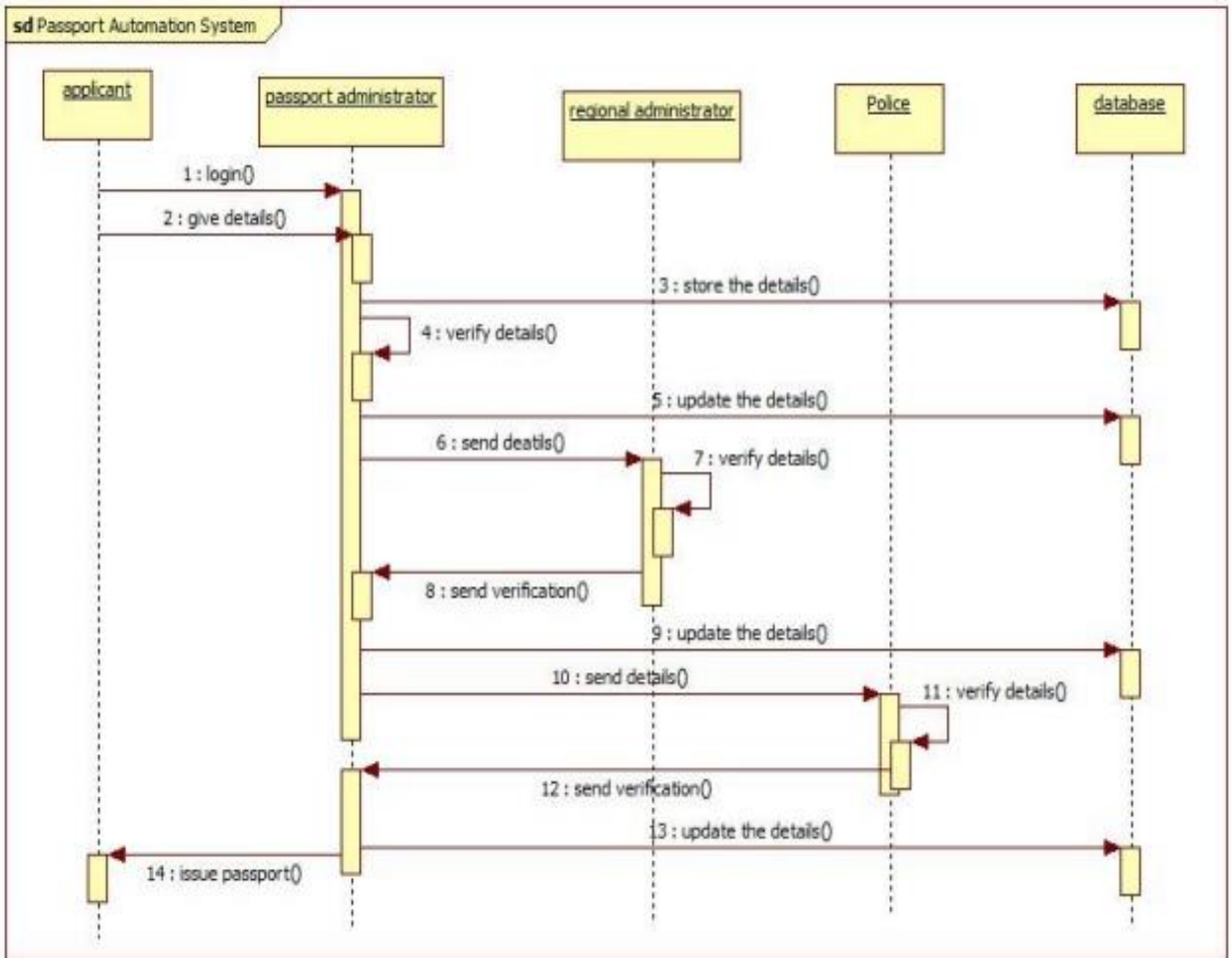
Use case Diagram

PAS --→





Sequence Diagram



Activity Diagram